

Zentrales OTA-Update-Management-System im Software-defined Vehicle

Architektur, Protokolle und Szenarien für OTA-Update und der
Software-defined Vehicle (SDV)-Architektur

Stephan Epp

5. März 2026

Inhaltsverzeichnis

1	Einleitung	3
1.1	SDV	3
1.2	Problemstellung	4
1.3	Ziele der Arbeit	5
2	Grundlagen	6
2.1	AUTOSAR Classic und Adaptive	6
2.2	Over-the-Air-Updates in der Automobilindustrie	7
2.3	Relevante AUTOSAR-Dienste für OTA	7
3	Das Software-defined Vehicle (SDV) als Paradigmenwechsel	8
3.1	Definition und Kernkonzept des SDV	8
3.2	E/E-Architekturtrends: Von der verteilten zur zentralisierten Architektur	9
3.2.1	Die Zonenarchitektur als strukturelles Rückgrat des SDV	9
3.3	High-Performance Computer (HPC) als zentrales Element des SDV	10
3.3.1	Android Automotive OS (AAOS) im SDV-Kontext	11
3.4	AUTOSAR Adaptive Plattform im SDV	11
3.5	OTA-Update im SDV: Anforderungen und Herausforderungen	12
3.5.1	Heterogenität der Plattformen	12
3.5.2	Kryptographische Leistungsanforderungen	12
3.5.3	Startzeit und Bootsequenz	13
3.5.4	Fahrzeugweite Konsistenz bei verteilter Software	13
3.6	SDV und Cybersicherheit: ISO 21434 im OTA-Kontext	13
3.7	Das SDV-Prinzip der Software-Verteilung: Funktionale Dekomposition	14
3.8	Praxisbeispiel: EVCC-Integration in die Zonenarchitektur	15
3.9	SDV als Befähiger: Zukünftige OTA-Szenarien	15
4	Vorgeschlagene Architekturerweiterung	16
4.1	Überblick	16
4.2	Zentralsteuergerät (ZSG)	16

4.3	Peripheriesteuergeräte (PSGs)	17
4.4	AUTOSAR-Softwarekomponenten der Erweiterung	17
4.5	Integration in das AUTOSAR-Schichtenmodell	19
5	Kommunikationsprotokolle	20
5.1	Backend-zu-ZSG-Kommunikation	20
5.2	ZSG-zu-PSG-Kommunikation	21
5.3	Nachrichtenformat des OTA-Update-Protokolls	21
6	Update-Szenarien	22
6.1	Zustandsautomat des ZSG-Update-Managers	22
6.2	Zeitgestaffeltes Update-Szenario	23
6.2.1	Phasen des Szenarios	23
6.3	Sequenzdiagramm	24
6.4	Fehlerbehandlung und Rollback	24
7	Sicherheitsaspekte	25
7.1	Bedrohungsmodell	25
7.2	Kryptographische Maßnahmen	26
7.3	Zuverlässigkeit und Ausfallsicherheit	26
8	Evaluation	27
8.1	Vergleich mit bestehenden Ansätzen	27
8.2	Laufzeitanalyse	27
9	Schluss	28
9.1	Zukünftige Arbeiten	28
9.2	Fazit	29
9.3	Ausblick	29
9.3.1	Prototypische Implementierung und experimentelle Validierung	29
9.3.2	Formale Verifikation und Sicherheitsnachweise	30
9.3.3	Künstliche Intelligenz und adaptives Update-Scheduling	31
9.3.4	Redundanz, Hochverfügbarkeit und Safety-Architektur	31
9.3.5	Post-Quanten-Kryptographie und langfristige Sicherheit	32
9.3.6	Feature-on-Demand, Geschäftsmodelle und regulatorische Einbettung	33
	Literaturverzeichnis	34

1 Einleitung

Der **AUTOSAR-Standard** (*AUTomotive Open System ARchitecture-Standard*) bildet seit seiner Einführung die technische Grundlage für die Softwarearchitektur moderner Kraftfahrzeuge. Parallel dazu vollzieht die Automobilindustrie gegenwärtig einen der tiefgreifendsten Paradigmenwechsel ihrer Geschichte: den Übergang zum **Software-defined Vehicle (SDV)**. Dieses Konzept – und seine weitreichenden Konsequenzen für OTA-Update-Systeme – stehen im Mittelpunkt der vorliegenden Arbeit.

1.1 SDV

Das **Software-defined Vehicle (SDV)** bezeichnet ein Fahrzeugkonzept, in dem Fahrzeugfunktionen nicht mehr primär durch spezialisierte, unveränderliche Hardware realisiert werden, sondern durch Software, die auf einer konsolidierten Rechenplattform läuft und über den gesamten Fahrzeuglebenszyklus hinweg aktualisiert, erweitert und angepasst werden kann [15]. Der Begriff wurde maßgeblich durch Tesla geprägt, das erstmals demonstrierte, dass Fahrzeugeigenschaften wie Reichweite, Beschleunigung oder Fahrerassistenzfunktionen *nach der Produktion* per drahtlosem Software-Update verändert werden können. Inzwischen verfolgen nahezu alle Tier-1-Automobilhersteller und etablierte Zulieferer diesen Ansatz systematisch [10].

Worin besteht der Paradigmenwechsel? In einem klassischen Fahrzeug des frühen 21. Jahrhunderts war jede Fahrzeugfunktion – Motorsteuerung, Bremsenregelung, Klimaautomatik, Infotainment – in einem dedizierten Steuergerät (**Electronic Control Unit (ECU)**) verankert. Funktionsänderungen erforderten entweder einen Werkstattbesuch (Flashen mittels Diagnosetool über OBD-Schnittstelle) oder sogar einen Hardware-Tausch. Das **SDV** löst diese enge Hardware-Funktions-Bindung auf: Software-Bausteine werden von der konkreten Siliziumgeneration entkoppelt und können auf leistungsstarken, zentralen *High-Performance Computern* (**High-Performance Computers (HPCs)**) konsolidiert werden. Ein modernes **SDV**-Fahrzeug kommt mit einem Bruchteil der früher üblichen ECU-Anzahl aus – bei gleichzeitig erheblich größerem Funktionsumfang [1].

Die Rolle der Softwaremenge. Moderne Premium-Fahrzeuge enthalten bereits heute mehr als **100 Millionen Zeilen Quellcode** – mehr als ein modernes Kampfflugzeug. Diese Softwaremenge wächst mit zunehmender Autonomisierung und Vernetzung weiter. Im **SDV** ist diese Software nicht mehr statisch: Sicherheitsrelevante Patches müssen innerhalb enger Fristen ausgerollt werden, neue Assistenzfunktionen werden als *Feature-on-Demand* nach Kauf freigeschaltet, und Diagnosedaten ermöglichen prädiktive Wartung auf Flottenebene. All dies setzt eine zuverlässige, fahrzeugweite **OTA-Update-Infrastruktur** voraus.

E/E-Architektur im SDV: Zonen statt Domänen. Der strukturelle Schlüssel des **SDV** ist der Übergang von der klassischen *domänenbasierten* zu einer *zonenbasierten* Elektrik/Elektronik-Architektur (**Elektrik/Elektronik (E/E)**). Während in der domänenbasierten Architektur Steuergeräte nach Funktion (Antrieb, Fahrwerk, Karosserie, Infotainment) geclustert wurden, organisiert die Zonenarchitektur die Fahrzeugelektronik nach *geographischer Lage* im Fahrzeug (Vorne-Links, Vorne-Rechts, Hinten, Dach). Jede Zone wird durch ein *Zonensteuergerät* (Zone Controller) koordiniert, das als intelligenter Hub für alle lokalen Sensoren und Aktuatoren dient. Die rechenintensiven Aufgaben – Sensorfusion,

Fahrfunktionsberechnungen, Update-Koordination – werden auf zentrale [HPCs](#) verlagert, die mit leistungsstarken [System-on-Chips \(SoCs\)](#) und POSIX-Betriebssystemen (Linux, [Android Automotive OS \(AAOS\)](#)) betrieben werden. Dieser Architekturwandel reduziert die Kabelbaummasse erheblich, vereinfacht die Hardware-Plattformstrategie und schafft die technische Grundlage dafür, dass ein einziger Update-Eintrittspunkt das gesamte Fahrzeugnetzwerk versorgen kann.

SDV und AUTOSAR: Chance und Herausforderung. AUTOSAR – als der dominierende Softwarestandard der Fahrzeugelektronik – reagierte auf den SDV-Trend mit der Einführung der **AUTOSAR Adaptive Platform** (ab Release 17-03). Diese ermöglicht dynamische Dienste, serviceorientierte Kommunikation über [Scalable service-Oriented MiddlewarE over IP \(SOME/IP\)](#) und [Data Distribution Service \(DDS\)](#) sowie ein integriertes Update-and-Configuration-Management ([Update and Configuration Management \(UCM\)](#)). Gleichzeitig bleibt die bewährte **AUTOSAR Classic Platform** für ressourcenbeschränkte Echtzeit-SGs im Einsatz – ein Fahrzeug enthält heute typischerweise *beide* Plattformen parallel. Diese Koexistenz heterogener AUTOSAR-Varianten auf derselben Fahrzeugelektronik stellt OTA-Update-Systeme vor besondere Herausforderungen: Klassische Mikrocontroller-SGs ohne POSIX-Betriebssystem müssen genauso zuverlässig aktualisiert werden wie leistungsstarke Adaptive-Plattform-Knoten, und die Konsistenz aller Softwareversionen im Fahrzeug muss zu jedem Zeitpunkt garantiert werden.

Motivation dieser Arbeit. Klassische AUTOSAR-Architekturen sehen eine direkte Verbindung zwischen einem Backend-Server und jedem einzelnen Steuergerät vor, was zu erhöhtem Kommunikationsaufwand, Synchronisierungsproblemen und mangelnder Fehlertoleranz führt [16]. Im [SDV](#) erfordert die zonenbasierte [E/E](#)-Architektur jedoch neue Koordinationsmechanismen, die weit über die bisherigen AUTOSAR-UCM-Konzepte hinausgehen. Diese Arbeit schlägt daher eine *hierarchische Erweiterung des AUTOSAR-Standards* vor, in der ein *Zentralsteuergerät (ZSG)* als dedizierter OTA-Gateway und Update-Treuhänder fungiert. Das ZSG empfängt Software-Pakete aus dem Backend, validiert und speichert sie („Parken“), und überträgt sie zeitgestaffelt und zustandsabhängig an *Peripheriesteuergeräte (PSG)* – darunter Bremsen-SGs, Radar-SGs und Türsteuerungs-SGs. Die Arbeit erweitert das bestehende AUTOSAR-Schichtenmodell um die notwendigen Dienste, definiert neue AUTOSAR-Softwarekomponenten und skizziert ein vollständiges Aktualisierungsszenario mit formalen Zustandsautomaten und Kommunikationssequenzdiagrammen.

Die Notwendigkeit, Fahrzeugsoftware während der gesamten Fahrzeuglebensdauer aktuell und sicher zu halten, hat zur Etablierung von *Firmware-over-the-Air* (FOTA) und *Software-over-the-Air* (SOTA) als Kerntechnologien des [SDV](#) geführt [2]. Während AUTOSAR Adaptive (R17 ff.) OTA-Updates prinzipiell adressiert, fehlt ein konsistentes, fahrzeugweites Koordinationsmodell, das die gesamte Kette vom Backend bis zum individuellen Steuergerät abdeckt. Dieses Defizit tritt im [SDV](#) besonders deutlich hervor, da hier [HPCs](#), Zonensteuergeräte und ressourcenbeschränkte Mikrocontroller als heterogene Plattform koexistieren.

1.2 Problemstellung

Der Übergang zum [SDV](#) verschärft strukturelle Schwächen bestehender OTA-Update-Architekturen in einem Maß, das einen grundlegenden Redesign erfordert. Die Probleme lassen sich in fünf Kategorien gliedern:

- **Direkte Backend-ECU-Kommunikation ohne Koordination:** In klassischen AUTOSAR-Architekturen verwaltet jedes Steuergerät seine eigene Backend-Verbindung. In einem SDV-Fahrzeug mit zonenbasierter Architektur und bis zu 60 Steuergeräten führt dies zu exzessivem Kommunikationsaufwand, unkontrollierter Bandbreitennutzung und einer enorm vergrößerten Angriffsfläche [10]. Ressourcenbeschränkte Classic-AUTOSAR-SGs (Bremsen-SG, Radar-SG) verfügen zudem weder über einen eigenen Mobilfunk-Stack noch über die kryptographische Leistungsfähigkeit, eine direkte TLS-Verbindung zum Backend zu unterhalten.
- **Fehlende fahrzeugweite Synchronisierung:** Das SDV zeichnet sich durch tief verwobene, funktionale Abhängigkeiten zwischen Steuergeräten aus: Ein Radar-SG-Update setzt eine kompatible Firmware-Version auf dem zentralen ADAS-Verarbeitungs-HPC voraus; ein Bremsenregelungs-Update darf nur zusammen mit einem kompatiblen Fahrdynamik-SG-Update aktiviert werden. Bestehende AUTOSAR-UCM-Konzepte sind primär auf das einzelne Steuergerät ausgerichtet und bieten keine Mechanismen für atomare, fahrzeugweite Transaktionen. Inkonsistente Software-Versionskombinationen können dabei sowohl sicherheitskritische Funktionsausfälle als auch schwer diagnostizierbare Interoperabilitätsprobleme verursachen.
- **Unzureichende Rollback-Mechanismen:** Im SDV mit heterogener Plattform – mehrere klassische Mikrocontroller-SGs neben leistungsstarken Adaptive-Plattform-Knoten – reicht ein Einzel-SG-Rollback nicht aus. Ein fehlerhaftes Update eines verbundenen Subsystems kann eine Kettenreaktion auslösen, die mehrere Steuergeräte in einen inkonsistenten Zustand versetzt. Bestehende Lösungen fehlt ein koordinierter *Fahrzeug-Rollback*, der alle betroffenen SGs atomar auf ihre Vor-Update-Versionen zurückführt.
- **Heterogenität der Plattformen:** Das SDV kombiniert POSIX-basierte HPCs (AUTOSAR Adaptive Platform, Linux, AAOS) mit klassischen Echtzeit-Mikrocontrollern (AUTOSAR Classic Platform), die weder über ein Dateisystem noch über einen Prozess-Manager verfügen. Ein einheitliches Update-Protokoll muss beide Welten bedienen: Das Staging eines Container-Images auf einem Adaptive-Dienst unterscheidet sich grundlegend vom Flash-Schreiben auf einem Classic-SG über UDS/SOME/IP. Bestehende Standards adressieren diese Heterogenität nicht durchgängig.
- **Sicherheitsanforderungen im SDV-Kontext:** Der erweiterte Connectivity-Footprint des SDV – permanente Backend-Verbindung, V2X-Kommunikation, Ladeleitungskommunikation – macht das Fahrzeug zu einem attraktiveren Angriffsziel als je zuvor. ISO 21434 und UN-Regelung Nr. 156 (CSMS/SUMS) verpflichten Hersteller zu nachweisbaren, end-to-end-gesicherten Update-Ketten. Gleichzeitig sind viele ältere Classic-AUTOSAR-SGs nicht für Secure On-Board Communication (SecOC) oder Transport Layer Security (TLS) ausgerüstet und benötigen einen sicherheitsaufteilenden Vermittler [11].

1.3 Ziele der Arbeit

Aus der in Kapitel 1.2 beschriebenen Problemlage ergibt sich ein klares Anforderungsprofil. Diese Arbeit verfolgt fünf aufeinander aufbauende Ziele:

1. **AUTOSAR-konforme Architekturenerweiterung für das SDV:** Entwurf eines Zentralsteuergeräts (ZSG) als dedizierter OTA-Gateway, der die Dual-Plattform-Heterogenität

des **SDV** (Classic + Adaptive) transparent überbrückt und vollständig in das AUTOSAR-Softwarekomponenten- und Dienstmodell integriert ist. Das ZSG soll dabei die Leistungsfähigkeit des fahrzeuginternen **HPC** nutzen, ohne die Autonomie und Sicherheitszertifizierung der angebundenen **Peripheriesteuergeräts (PSGs)** zu kompromittieren.

2. **Spezifikation der fahrzeuginternen Kommunikationsprotokolle:** Definition der Nachrichten, Sequenzen und Fehlermodi für die Kommunikation zwischen ZSG und heterogenen **PSGs**, unter Berücksichtigung der im **SDV** eingesetzten Bussysteme (CAN-FD, Automotive Ethernet, **SOME/IP**) sowie der kryptographischen Absicherung durch **SecOC** und **TLS**.
3. **Definition eines fahrzeugweiten Zustandsautomaten:** Formale Spezifikation des gesamten Update-Lebenszyklus vom Backend-Download bis zur abschließenden Validierung, inklusive koordinierter Rollback-Sequenz für den Fall, dass ein oder mehrere **PSGs** die Aktivierung nicht erfolgreich abschließen. Der Zustandsautomat muss **Automotive Safety Integrity Level (ASIL)**-konforme Update-Fenster einhalten und den simultanen Betrieb sicherheitskritischer Fahrzeugfunktionen während des Update-Prozesses garantieren.
4. **Entwicklung eines zeitgestaffelten Update-Szenarios:** Ausarbeitung und Analyse eines vollständigen, realistischen Multi-SG-Update-Szenarios für ein **SDV**-Fahrzeug mit zonenbasierter Architektur, das Bremsen-SG (**ASIL-D**), Radar-SG (**ASIL-B**) und Tür-SG (**ASIL-A**) umfasst. Die Laufzeitanalyse quantifiziert Overhead durch kryptographische Operationen, interne Übertragungszeiten auf den jeweiligen Fahrzeugbussen sowie die Gesamtdauer inkl. Rollback-Pfad.
5. **Bewertung der Sicherheits- und Zuverlässigkeitseigenschaften:** Systematische Analyse des vorgeschlagenen Systems gegen die Bedrohungsklassen aus **Threat Analysis and Risk Assessment (TARA)** (ISO 21434), Nachweis der Konformität mit den strukturellen Anforderungen von UN-R 156 (SUMS) sowie vergleichende Einordnung gegenüber bestehenden proprietären und standardisierten OTA-Ansätzen.

Abgrenzung. Diese Arbeit konzentriert sich auf die *Architektur* und *Protokolle* des Update-Systems. Die Implementierung konkreter AUTOSAR-Softwarekomponenten in C/C++, die Entwicklung eines HiL-Testaufbaus sowie die formale Verifikation der Zustandsautomaten sind als explizite Weiterführungsthemen benannt (Kapitel 9.3) und liegen außerhalb des Rahmens dieser Arbeit.

2 Grundlagen

2.1 AUTOSAR Classic und Adaptive

AUTOSAR (*AUTomotive Open System ARchitecture*) ist ein weltweiter Entwicklungsstandard für automobile Software, der von einem Konsortium aus Automobilherstellern, Zulieferern und Werkzeuganbietern entwickelt wurde. Der Standard gliedert sich in zwei Hauptvarianten:

AUTOSAR Classic Platform (CP): Für ressourcenbeschränkte, harte Echtzeit-Steuergeräte. Definiert ein statisches Laufzeitsystem (RTE), Basissoftware-Module (BSW) und standardisierte Schnittstellen.

AUTOSAR Adaptive Platform (AP): Für leistungsstarke, POSIX-basierte Steuergeräte, die dynamische Dienste, serviceorientierte Kommunikation (SOME/IP) und Update-und-Konfigurations-Management (UCM) unterstützen.

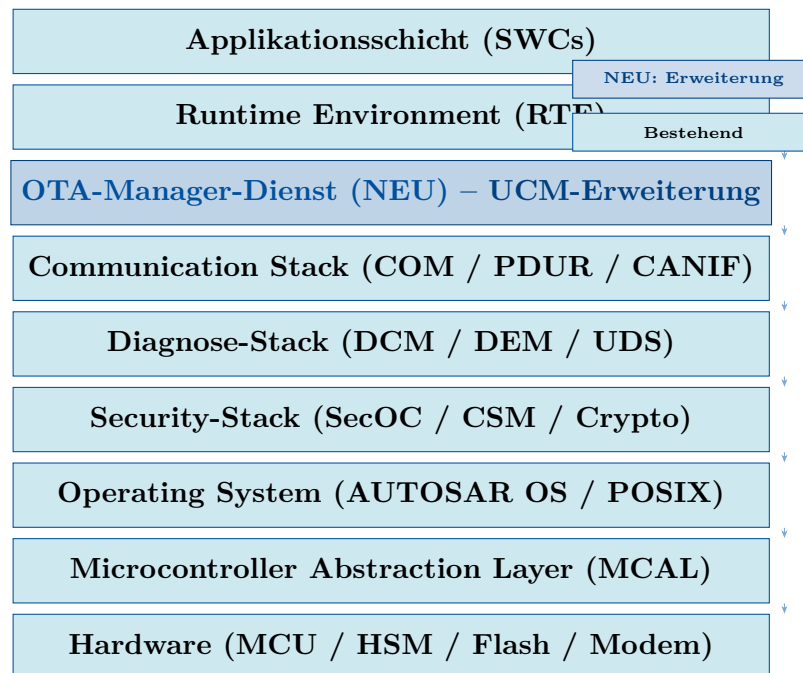


Abbildung 1: AUTOSAR-Schichtenmodell (Classic Platform, vereinfacht). Die vorgeschlagene Erweiterung fügt den *OTA-Manager-Dienst* zwischen Laufzeitsystem und Kommunikationsstack ein.

2.2 Over-the-Air-Updates in der Automobilindustrie

OTA-Updates bezeichnen das drahtlose Aufspielen von Software-Paketen auf Fahrzeugsteuergeräte. Man unterscheidet:

- **FOTA (Firmware OTA):** Aktualisierung von Firmware auf ressourcenbeschränkten Classic-AUTOSAR-SGs.
- **SOTA (Software OTA):** Aktualisierung von Applikationssoftware auf leistungsstärkeren Adaptive-AUTOSAR-SGs.

Der Update-Prozess umfasst typischerweise folgende Phasen: Download → Verifikation → Staging → Installation → Aktivierung → Validierung.

2.3 Relevante AUTOSAR-Dienste für OTA

Der *Update and Configuration Management* (UCM) ist ein Dienst der AUTOSAR Adaptive Platform, der Software-Pakete verwaltet. Er bietet:

- Paketmanagement (Herunterladen, Speichern, Installieren)
- Abhängigkeitsprüfung

- Rollback-Unterstützung
- Authentifizierung über kryptographische Signaturen

In der bestehenden AUTOSAR-Architektur ist UCM jedoch primär für ein einzelnes Steuergerät konzipiert. Eine fahrzeugweite Koordination fehlt.

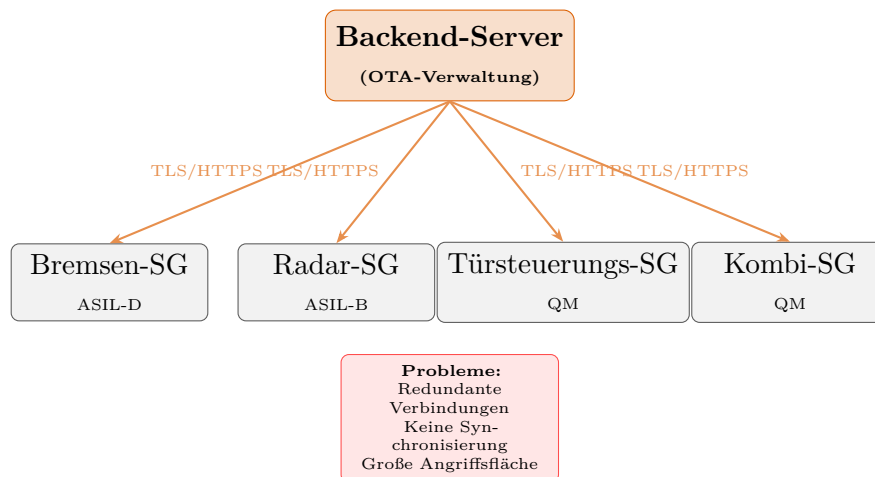


Abbildung 2: Konventioneller OTA-Ansatz: Jedes Steuergerät kommuniziert direkt mit dem Backend-Server (vereinfachte Darstellung). Nachteile: hohe Bandbreite, keine Synchronisierung, erhöhte Angriffsfläche.

3 Das Software-defined Vehicle (SDV) als Paradigmenwechsel

3.1 Definition und Kernkonzept des SDV

Das **Software-defined Vehicle (SDV)** bezeichnet ein Fahrzeugkonzept, bei dem Fahrzeugfunktionen nicht mehr primär durch spezialisierte, unveränderliche Hardware realisiert werden, sondern durch Software, die auf einer konsolidierten, leistungstarken Hardware-Plattform ausgeführt wird und über den gesamten Fahrzeuglebenszyklus hinweg aktualisiert, erweitert und angepasst werden kann [15]. Der Begriff wurde maßgeblich durch Tesla geprägt, das erstmals zeigte, dass Fahrzeugeigenschaften wie Beschleunigung, Reichweite oder Assistenzfunktionen nach der Produktion per Software-Update verändert werden können. Inzwischen verfolgen nahezu alle Automobilhersteller und Zulieferer diesen Ansatz [10].

Die fundamentale Verschiebung des SDV gegenüber klassischen Fahrzeugarchitekturen lässt sich in drei Dimensionen beschreiben:

Hardware-Konsolidierung: Anstatt für jede Fahrzeugfunktion ein dediziertes Steuergerät zu verwenden, werden Funktionen auf wenigen, hochleistungsfähigen Rechenplattformen (HPC, SoC) gebündelt. Ein modernes SDV-Fahrzeug kann mit wesentlich weniger ECUs auskommen als ein klassisches Fahrzeug, obwohl es mehr Funktionen bietet.

Software-Entkopplung: Software ist von der spezifischen Hardware-Generation weitgehend entkoppelt. Neue Funktionen, Bugfixes und Sicherheitsupdates können über

den gesamten Fahrzeuglebenszyklus hinweg per OTA-Update eingespielt werden, ohne physischen Werkstattbesuch [2].

Serviceorientierung: Fahrzeugfunktionen werden als Dienste (Services) modelliert, die über standardisierte Schnittstellen (z. B. SOME/IP, DDS) kommunizieren. Dies ermöglicht dynamische Konfiguration, Skalierung und den Austausch von Einzelkomponenten [11].

3.2 E/E-Architekturtrends: Von der verteilten zur zentralisierten Architektur

Die Fahrzeug-E/E-Architektur hat in den vergangenen Jahrzehnten eine deutliche Entwicklung durchlaufen. Table 1 fasst die wesentlichen Entwicklungsstufen zusammen.

Tabelle 1: Evolutionsstufen der E/E-Architektur im Automobil

Generation	Architekturprinzip	Typische Anzahl	ECU-OTA-Fähigkeit
1990er	Vollverteilt, domänenlos	10–30	Keine
2000er	Domänenbasiert (Antrieb, Fahrwerk ...)	30–80	Keine / Werkstatt
2010er	Domänenzentralisiert, Gateway	80–150	Teilweise (SOTA/FO-TA)
2020er	Zonenarchitektur + HPC	30–60	Vollständig (SDV)
2030+	Fahrzeugrechner-Architektur	<20	Nativ + KI-gestützt

3.2.1 Die Zonenarchitektur als strukturelles Rückgrat des SDV

Die für das SDV charakteristische **Zonenarchitektur** unterteilt das Fahrzeug in geographische Zonen (z. B. Vorne-Links, Vorne-Rechts, Hinten-Links, Hinten-Rechts, Dach) [16]. Jede Zone wird durch ein *Zonensteuergerät* (engl. *Zone Controller*) verwaltet, das als intelligenter Hub für alle Sensoren, Aktuatoren und kleineren Steuergeräte innerhalb seiner Zone dient. Die Zonenarchitektur bietet folgende Vorteile gegenüber der klassischen domänenbasierten Architektur:

- **Kabelreduktion:** Durch die geographische Organisation entfallen lange Kabelwege quer durch das Fahrzeug. Sensoren und Aktuatoren werden lokal an das nächste Zonensteuergerät angeschlossen.
- **Leistungskonzentration:** Rechenintensive Aufgaben werden auf zentrale HPCs verlagert, die mit leistungsstarken SoCs und POSIX-basierten Betriebssystemen (Linux, Android Automotive OS) ausgeführt werden.
- **Update-Vereinfachung:** Software-Updates können zentral auf den HPCs und Zonensteuergeräten eingespielt werden, anstatt jede einzelne ECU direkt ansprechen zu müssen.

- **Skalierbarkeit:** Neue Fahrzeugvarianten können durch Software- Konfiguration der Zonensteuergeräte realisiert werden, ohne neue Hardware zu entwickeln.

Die folgende TikZ-Darstellung zeigt schematisch die Zonenarchitektur eines modernen SDV-Fahrzeugs mit zwei zentralen HPCs und vier Zonensteuergeräten:

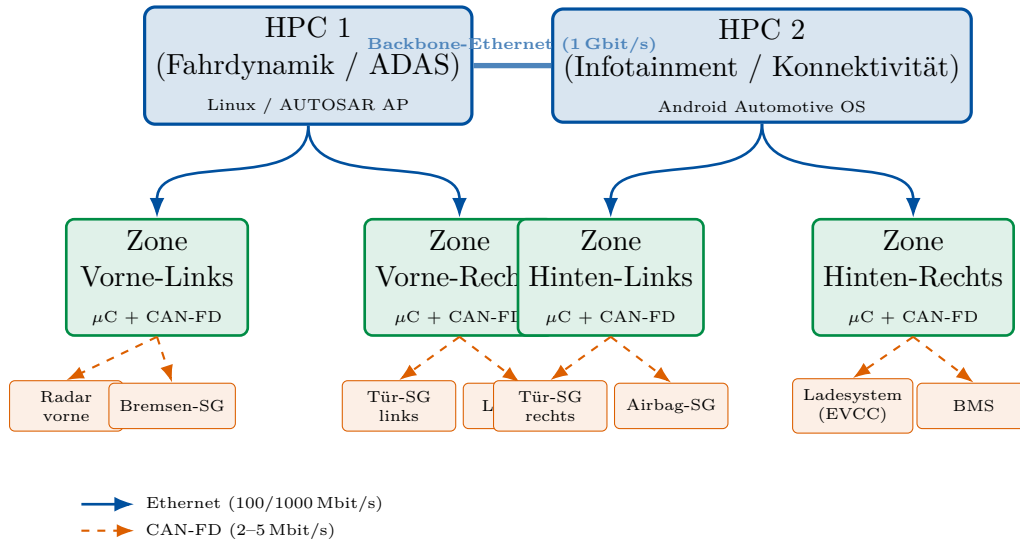


Abbildung 3: Schematische Zonenarchitektur eines SDV-Fahrzeugs mit zwei zentralen HPCs und vier Zonensteuergeräten. OTA-Updates werden über die HPCs empfangen und über die Zonensteuergeräte an die angebundenen ECUs verteilt.

3.3 High-Performance Computer (HPC) als zentrales Element des SDV

Der High-Performance Computer (HPC) ist das Herzstück des SDV. Er ist typischerweise als leistungsstarkes System-on-Chip (SoC) ausgeführt und kombiniert:

- Mehrere ARM-Cortex-A-Kerne (oft 8–16 Kerne bei 2–3 GHz) für applikationsseitige Aufgaben
- Dedizierte Echtzeit-Kerne (ARM-Cortex-R oder -M) für sicherheitskritische und zeitkritische Funktionen
- Hardware-Beschleuniger für maschinelles Lernen (NPU/GPU) für Fahrerassistenz- und Kamerafunktionen
- Kryptographische Hardware-Beschleuniger und Hardware Security Module (HSM) für Sicherheitsfunktionen einschließlich OTA-Verifikation
- Mehrere Netzwerkcontroller für Ethernet (100 Mbit/s bis 10 Gbit/s), CAN-FD, LIN und Mobilfunk (4G/5G)

Betriebssystemseitig kommen auf HPCs im SDV hauptsächlich POSIX-basierte Systeme zum Einsatz: Linux (z. B. Yocto-basiert), Android Automotive OS (AAOS) oder hypervisor-basierte Ansätze, bei denen ein Echtzeit-Betriebssystem (RTOS) und ein General-Purpose-OS (GPOS) parallel ausgeführt werden. Dies ermöglicht die gleichzeitige Erfüllung von Echtzeit-Anforderungen und die Nutzung moderner Software-Ökosysteme.

3.3.1 Android Automotive OS (AAOS) im SDV-Kontext

Android Automotive OS hat sich als führendes Infotainment-Betriebssystem im SDV etabliert und findet zunehmend auch für sicherheitsnachrüstungsrelevante Dienste Verwendung [16]. AAOS ist eine vollständige Android-Variante, die direkt auf Fahrzeughardware läuft (nicht auf einem angeschlossenen Smartphone). Für OTA-relevante Dienste ist die **Partition-Architektur** von AAOS entscheidend:

- **System-Partition:** Enthält das Android-Kernsystem mit den höchsten Privilegien. Hier laufen systemnahe Dienste und das Android-Framework. Ist nicht für OEM-spezifische Erweiterungen vorgesehen.
- **Vendor-Partition:** Enthält OEM- und Zulieferer-spezifische Treiber und Dienste. Hat Zugriff auf alle notwendigen System-APIs und bietet Schutzmechanismen gegen unbefugte Änderungen. Dies ist die geeignete Partition für fahrzeugspezifische OTA-Dienste.
- **Product-Partition:** Enthält Endbenutzer-Apps und OEM-Anpassungen. Wird über das Vendor Interface von der Vendor-Partition getrennt.

Für den in dieser Arbeit vorgeschlagenen ZSG-OTA-Manager bedeutet dies: Läuft der ZSG auf einem HPC mit AAOS, so ist der `OTA_UpdateManager_SWC` als nativer Dienst in der **Vendor-Partition** zu implementieren. Er hat damit vollen Zugriff auf Netzwerk-APIs, Kryptographie-APIs und das Android Keystore System, während gleichzeitig Isolation von Nutzer-Apps sichergestellt ist. Der Android Service Manager übernimmt dabei automatisch die Überwachung und den Neustart des Dienstes im Fehlerfall [16].

3.4 AUTOSAR Adaptive Platform im SDV

Die AUTOSAR Adaptive Platform (AP) wurde gezielt für den SDV-Kontext entwickelt und stellt damit das Software-Framework der Wahl für HPC-basierte Steuergeräte dar. Im Unterschied zur Classic Platform unterstützt die AP:

- **Dynamische Service-Erkennung:** Dienste registrieren sich dynamisch und werden zur Laufzeit gefunden (Service Discovery via SOME/IP oder DDS), anstatt statisch zur Compile-Zeit konfiguriert zu werden.
- **Flexible Software-Verteilung:** Die AP unterstützt nativ das Ausführen von Software in Containern (z. B. Docker-ähnliche Isolation), was das Deployment und Update einzelner Dienste unabhängig voneinander ermöglicht.
- **Update and Configuration Management (UCM):** Die AP definiert den UCM-Dienst als Standardmechanismus für Software-Updates. Im SDV-Kontext muss UCM jedoch fahrzeugweit koordiniert werden, was der zentrale Beitrag dieser Arbeit adressiert.
- **POSIX-basierte Ausführungsumgebung:** Entwickler können Standard- C++-Bibliotheken und Linux-Werkzeuge nutzen, was die Softwareentwicklung erheblich vereinfacht im Vergleich zur Classic Platform.

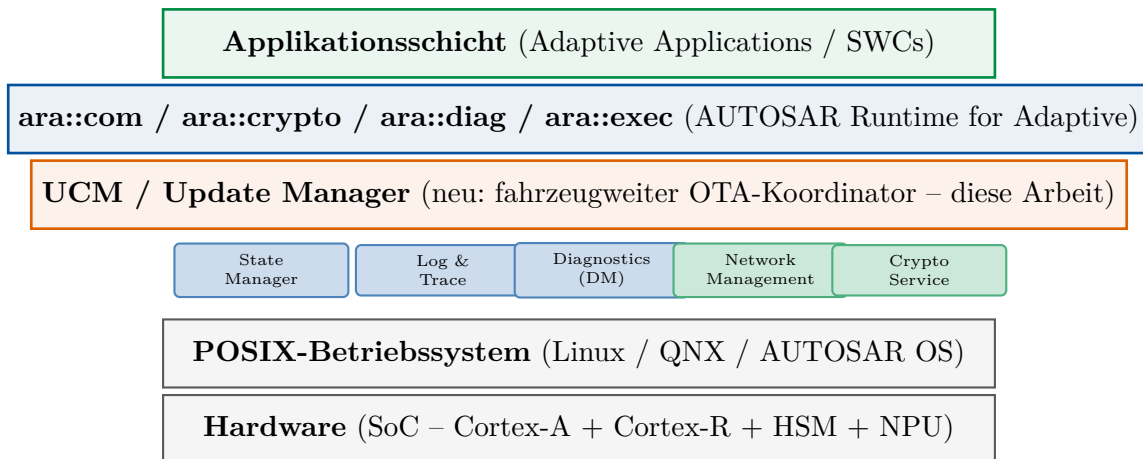


Abbildung 4: AUTOSAR Adaptive Platform im SDV-Kontext. Die hervorgehobene Schicht zeigt die in dieser Arbeit vorgeschlagene Erweiterung des UCM um einen fahrzeugweiten OTA-Koordinator.

3.5 OTA-Update im SDV: Anforderungen und Herausforderungen

Das SDV stellt gegenüber klassischen Fahrzeugarchitekturen deutlich höhere Anforderungen an OTA-Update-Systeme. Die folgende Analyse basiert auf den Erkenntnissen aus dem industriellen Umfeld, insbesondere dem Vorentwicklungsprojekt von Vector Informatik zur Integration des EVCC in die Zonenarchitektur [16].

3.5.1 Heterogenität der Plattformen

Im SDV koexistieren fundamental unterschiedliche Plattformen:

- **HPC / SoC (AUTOSAR Adaptive, Linux, AAOS):** Hohe Rechenleistung, großer Speicher, schnelle Netzwerkkinterfaces. Native OTA-Unterstützung via UCM oder Android OTA (A/B-Updates). Bootzeit: mehrere Sekunden bis Minuten.
- **Zonensteuergeräte (AUTOSAR Classic + Adaptive):** Mittlere Rechenleistung, eingeschränkter Flash-Speicher, CAN-FD oder Ethernet. Bootzeit: unter 500 ms. Sicherheitskritische Echtzeit-Anforderungen.
- **Periphere Mikrocontroller (AUTOSAR Classic):** Sehr geringe Rechenleistung (typisch 32-Bit ARM Cortex-M, 64–512 MB Flash), CAN oder LIN. Bootzeit: unter 100 ms. Oft ASIL-D-zertifiziert.

Der OTA-Update-Mechanismus muss alle drei Plattfortmtypen unterstützen und dabei jeweils die spezifischen Einschränkungen hinsichtlich Rechenleistung, Speicherkapazität und Netzwerkbandbreite berücksichtigen.

3.5.2 Kryptographische Leistungsanforderungen

Die im SDV eingesetzte ISO 15118-Ladekommunikation demonstriert exemplarisch das kryptographische Leistungsproblem: Ein TLS 1.3-Handshake mit elliptischen Kurven (ECD-SA, P-256) benötigt auf einem klassischen Mikrocontroller bei 80 MHz Taktfrequenz typisch

2–5 Sekunden [16]. Auf einem HPC mit Hardware-Beschleuniger dauert derselbe Vorgang unter 200 Millisekunden. Dies ergibt einen Laufzeit-Vorteil von Faktor 8–10, wie in realen Testszenarien bestätigt wurde.

Tabelle 2: Vergleich kryptographischer Laufzeiten: Mikrocontroller vs. HPC im SDV

Operation	Classic- μ C (80 MHz, SW)	HPC SoC (2 GHz, HW- Beschl.)	Faktor
SHA-256 (1 MB)	≈ 800 ms	< 5 ms	$> 160\times$
ECDSA-P256 Sign	≈ 1200 ms	< 2 ms	$> 600\times$
ECDSA-P256 Verify	≈ 2400 ms	< 4 ms	$> 600\times$
TLS 1.3 Handshake	2000–5000 ms	< 200 ms	10–25 $\times$
AES-256-GCM (1 MB)	≈ 400 ms	< 1 ms	$> 400\times$

Diese Leistungsunterschiede unterstreichen die Notwendigkeit, kryptographisch intensive OTA-Operationen auf HPCs zu verlagern, während sicherheitskritische, echtzeitfähige Funktionen auf Mikrocontrollern verbleiben. Genau dies ist das Prinzip des in Kapitel 4 vorgeschlagenen verteilten ZSG-Konzepts.

3.5.3 Startzeit und Bootsequenz

Ein oft übersehener Aspekt im SDV ist die unterschiedliche Bootzeit der verschiedenen Plattformen. Mikrocontroller starten in Millisekunden und sind damit sofort einsatzbereit. HPCs hingegen benötigen für den Start von Linux oder AAOS typisch 5–30 Sekunden. Dies führt zu einem strukturellen Designproblem: Das Fahrzeug braucht sofort nach dem Einschalten bestimmte Grundfunktionen (z. B. Ladekommunikation, Bremse), die keinen Verzug dulden, während die OTA-Kryptographie und komplexe Protokollverarbeitung auf dem HPC noch nicht verfügbar sind.

Die in diesem Kontext von Vector Informatik beschriebene Lösung [16] zeigt exemplarisch: Der Mikrocontroller übernimmt die ersten Schritte der Ladesequenz (SLAC, SDP) noch vor der Verfügbarkeit des HPC. Sobald der HPC hochgefahren ist, übernimmt dieser die kryptographisch anspruchsvollen Teile der Ladekommunikation. Dieses Prinzip der *gestaffelten Übernahme* muss auch für OTA-Updates gelten: Der Mikrocontroller initiiert den OTA-Handshake, der HPC führt die Verifikation und Kryptographie durch.

3.5.4 Fahrzeugweite Konsistenz bei verteilter Software

Im SDV stellt die Konsistenz der Software-Versionen über alle Steuergeräte hinweg eine besondere Herausforderung dar. Da Funktionen auf mehrere Steuergeräte verteilt sind, müssen Software-Updates atomisch auf allen beteiligten Geräten eingespielt werden. Ein partiell eingespieltes Update – bei dem beispielsweise der Algorithmus für eine ADAS-Funktion auf dem HPC aktualisiert ist, der zugehörige Sensor-Treiber auf dem Zonensteuergerät jedoch nicht – kann zu inkonsistentem und potenziell gefährlichem Verhalten führen [10].

3.6 SDV und Cybersicherheit: ISO 21434 im OTA-Kontext

Das SDV erweitert die Angriffsfläche eines Fahrzeugs erheblich. Durch die Konsolidierung auf wenige zentrale HPCs entstehen neue Risiken: Ein kompromittierter HPC

kann theoretisch auf alle angebundenen Steuergeräte zugreifen. Gleichzeitig bietet das SDV aber auch neue Schutzmaßnahmen durch die Nutzung moderner Betriebssystem-Sicherheitsmechanismen (SELinux, SecureEnclave, TrustZone).

Die ISO 21434 definiert als Rahmenwerk für Cybersicherheit im Automobil (*Cybersecurity Engineering*) einen risikobasierten Ansatz (TARA), der auch für OTA-Update-Systeme im SDV gilt. Besonders relevant sind im SDV-Kontext:

- **NAT-basierte Netzwerksegmentierung:** Zonensteuergeräte nutzen NAT-ähnliche Router, die nur erlaubte Protokolle zwischen externen Schnittstellen und dem internen Fahrzeugnetz durchlassen [16]. Dies schützt interne ECUs vor direkten Angriffen von außen.
- **Zertifikats-basierte Authentifizierung (PKI):** Im SDV werden alle OTA-Pakete durch digitale Signaturen (PKI, ISO 15118) authentifiziert. Zertifikate und private Schlüssel werden im HSM des HPC sicher verwahrt.
- **Netzwerk-Monitoring (IDS/IPS):** Der zentrale HPC kann als Intrusion Detection System fungieren und abnormale Kommunikationsmuster im Fahrzeugnetzwerk erkennen.

3.7 Das SDV-Prinzip der Software-Verteilung: Funktionale Dekomposition

Ein zentrales Entwurfsprinzip des SDV ist die *funktionale Dekomposition*: Software-Funktionen werden gemäß ihren nicht-funktionalen Anforderungen (Echtzeit, Rechenleistung, Sicherheitsniveau, Updatehäufigkeit) auf geeignete Hardware-Plattformen verteilt. Figure 5 illustriert dieses Prinzip für den OTA-Update-Anwendungsfall.

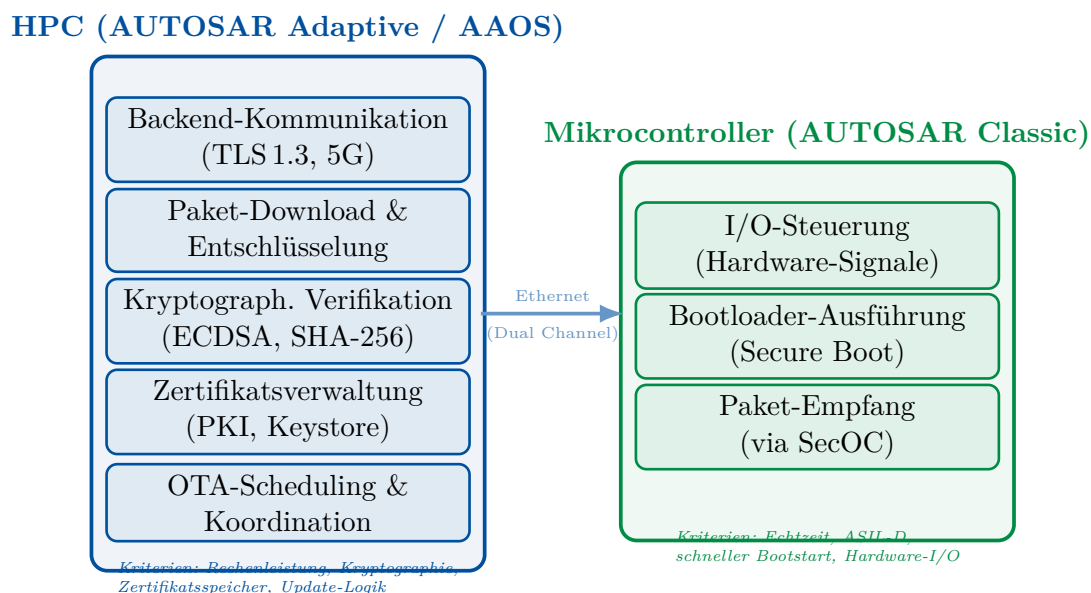


Abbildung 5: Funktionale Dekomposition der OTA-Funktionen im SDV nach dem Prinzip der nicht-funktionalen Anforderungen. Kryptographisch intensive Aufgaben verbleiben auf dem HPC; zeitkritische und sicherheitsrelevante Funktionen auf dem Mikrocontroller.

3.8 Praxisbeispiel: EVCC-Integration in die Zonenarchitektur

Als Referenzbeispiel für die SDV-Prinzipien im OTA-Kontext dient die Integration des Electric Vehicle Communication Controller (EVCC) in die Zonenarchitektur, wie sie von Tobias Schneider und Michael Vick bei Vector Informatik untersucht wurde [16].

Das EVCC veranschaulicht exemplarisch die Herausforderung, eine Funktion mit heterogenen Anforderungen in eine SDV-Zonenarchitektur zu integrieren: Einerseits umfasst es sicherheitskritische Hardware-Steuerung (Control Pilot, Verriegelungsmotor der Ladedose, CCS-Signale) gemäß ISO 26262; andererseits erfordert es ressourcenintensive Kryptographie (TLS 1.3, PKI-Zertifikatsverwaltung) gemäß ISO 15118-20 und ISO 21434.

Die von Vector untersuchte verteilte Lösung teilt das EVCC auf zwei Steuergeräte auf:

1. **Mikrocontroller-basiertes Gateway** (MICROSAR Classic): Übernimmt die sicherheitskritischen Funktionen (Control Pilot, Hardware-I/Os, SLAC, SDP, Verriegelungsmotor), profitiert von Echtzeitfähigkeit und schnellem Bootstart (< 100 ms). Implementiert einen NAT-ähnlichen Router, der nur Ladeprotokoll-Nachrichten in das Fahrzeugnetz durchlässt.
2. **HPC mit Android Automotive OS** (MICROSAR Charge): Übernimmt die rechenintensive Ladekommunikation (TLS 1.3, EXI-Encoding, XML-Security, Zertifikatsverwaltung für Plug & Charge). Der Charge-Service wird als nativer Dienst in der Vendor-Partition des AAOS implementiert und durch den Android Service Manager überwacht.

Das Ergebnis ist bemerkenswert: Die TLS 1.3-Laufzeit verbessert sich um den Faktor 8–10 gegenüber einer rein Mikrocontroller-basierten Lösung, alle ISO-15118-20-Zeitlimits werden eingehalten, und der Speicherbedarf für Zertifikatsketten stellt auf der SoC-Plattform kein Problem dar [16].

Übertragung auf das OTA-Update-System. Die Erkenntnisse aus der EVCC-Integration lassen sich direkt auf das in dieser Arbeit vorgeschlagene ZSG-OTA-System übertragen:

- Der **Mikrocontroller im ZSG** übernimmt den Bootvorgang und die ersten Handshake-Schritte des OTA-Protokolls noch vor der Verfügbarkeit des HPC. Er stellt sicher, dass das Fahrzeug auch während eines HPC-Neustarts (z. B. nach einem Update des AAOS) die Grundfunktionen aufrechterhält.
- Der **HPC im ZSG** übernimmt die kryptographische Verifikation aller OTA-Pakete, verwahrt Zertifikate und private Schlüssel im Hardware Keystore, und koordiniert die zeitgestaffelte Verteilung an die PSGs.
- Die **Dual-Channel-Ethernet-Verbindung** zwischen Mikrocontroller und HPC trennt das OTA-Protokoll von der Statussynchronisierung, analog zur EVCC-Architektur.

3.9 SDV als Befähiger: Zukünftige OTA-Szenarien

Das SDV eröffnet OTA-Update-Szenarien, die mit klassischen Fahrzeugarchitekturen nicht realisierbar wären:

Differenzielle Funktions-Updates: Statt vollständiger Firmware-Images können im SDV einzelne Software-Dienste (Microservices) unabhängig aktualisiert werden. Ein neuer Parkassistent kann als Dienst auf dem HPC eingespielt werden, ohne andere Fahrzeugfunktionen zu beeinträchtigen.

Feature-on-Demand: Fahrzeugfunktionen können nach dem Kauf freigeschaltet werden (z. B. erhöhte Motorleistung, erweiterte Assistenzfunktionen). Dies erfordert sichere OTA-Mechanismen für Lizenzierung und Aktivierung.

KI-modellbasierte Updates: KI-Netzwerkmodelle für Fahrerassistenz und Sprachsteuerung können via OTA aktualisiert werden. Die Paketgrößen können dabei Gigabytes erreichen, was neue Anforderungen an Bandbreite, Speicherverwaltung und Delta-Update-Algorithmen stellt.

Multi-ECU-Transaktionen: Ein funktionales Update (z. B. neues ABS-Steuergesetz) betrifft gleichzeitig HPC, Zonensteuergerät und Bremsen-SG. Im SDV ermöglicht der zentrale ZSG-OTA-Manager atomare Multi-ECU-Transaktionen mit vollständigem Rollback bei Teilversagen.

Continuous Deployment (CD) im Fahrzeug: In Entwicklungsfahrzeugen und in der Flottenverwaltung ermöglicht das SDV Continuous-Integration/Continuous-Deployment-Pipelines (CI/CD), bei denen neue Software-Versionen automatisch getestet und eingespielt werden – analog zu modernen Cloud-Deployments.

4 Vorgeschlagene Architekturerweiterung

4.1 Überblick

Der Kernvorschlag dieser Arbeit ist die Einführung eines **Zentralsteuergeräts (ZSG)** – im Englischen *Central Update Gateway ECU (CUGE)* – das als einziger Kommunikationspartner des Backend-Servers agiert und alle Update-Operationen im Fahrzeug koordiniert.

4.2 Zentralsteuergerät (ZSG)

Das ZSG ist ein dediziertes, leistungsstarkes Steuergerät auf Basis der **AUTOSAR Adaptive Platform**. Es übernimmt folgende Funktionen:

1. **Backend-Kommunikation:** Einzige TLS-gesicherte Außenverbindung über ein Mobilfunkmodem (4G/5G).
2. **Paket-Download:** Herunterladen und Zwischenspeichern (*Parken*) von Update-Paketen für alle PSGs.
3. **Kryptographische Verifikation:** Signaturprüfung und Integritätsprüfung aller Pakete.
4. **Update-Koordination:** Zeitgestaffelte Verteilung der Pakete an die PSGs gemäß einem konfigurierbaren Update-Plan.

Tabelle 3: SDV-Merkmale und deren Auswirkungen auf das OTA-Update-System

SDV-Merkmal	OTA-Anforderung	Lösungsansatz (diese Arbeit)
Zonenarchitektur	Hierarchische Update-Verteilung über Zonensteuergeräte	ZSG als zentraler Koordinator mit gestaffelter Verteilung
HPC mit POSIX-OS	Native Kryptographie und PKI-Verwaltung	OTA-Manager in Vendor-Partition (AA-OS)
Heterogene Plattformen	FOTA und SOTA in einem Update-Zyklus	Einheitliches SUM-Manifest mit plattform-spezifischen Paketen
Schneller Bootstart μ C	OTA-Handshake vor HPC-Verfügbarkeit	Gestaffelter Bootstart: μ C übernimmt erste Schritte
Serviceorientierung	Update einzelner Dienste ohne Systemstopp	Modulare SWC-Architektur mit unabhängigen Paket-IDs
5G-Konnektivität	Hohe Bandbreite für große Update-Pakete	Backend-Download exklusiv über ZSG mit 5G-Modem
ISO 21434 (Cybersicherheit)	Authentifizierung aller Update-Pakete	PKI-Signaturen, TLS 1.3, HSM-basierte Schlüsselverwaltung
Mehrere Ladedosen (LKW/MCS)	Zentrales Update aller EVCC-Instanzen	Einmalige Aktualisierung des Charge-Service im HPC

5. **Statusüberwachung:** Monitoring aller PSG-Update-Zustände.

6. **Rollback-Management:** Koordinierter Rollback im Fehlerfall.

4.3 Peripheriesteuergeräte (PSGs)

Die PSGs können sowohl auf der AUTOSAR Classic Plattform (ressourcenbeschränkte SGs wie Bremsen-SG, Türsteuerungs-SG) als auch auf der AUTOSAR Adaptive Plattform (leistungsstärkere SGs wie Radar-SG) basieren.

4.4 AUTOSAR-Softwarekomponenten der Erweiterung

Die vorgeschlagene Erweiterung führt folgende neue AUTOSAR Softwarekomponenten (SWCs) ein:

OTA_UpdateManager_SWC (ZSG) Hauptkomponente des ZSG. Verwaltet den gesamten Update-Lebenszyklus:

- Empfang und Parsing von *Software Update Manifest* (SUM)-Dateien

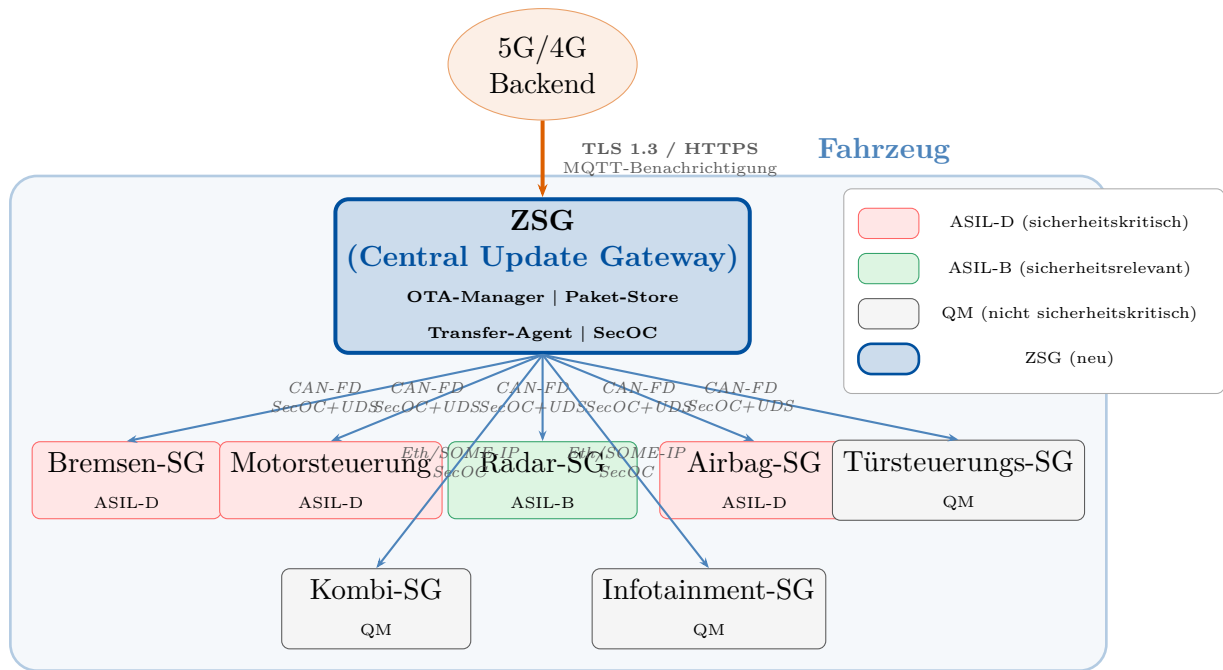


Abbildung 6: Überblick der vorgeschlagenen Architektur mit Zentralsteuergerät (ZSG) und angeordneten Peripheriesteuergeräten (PSGs). Das ZSG ist der einzige Kommunikationsknoten zum Backend.

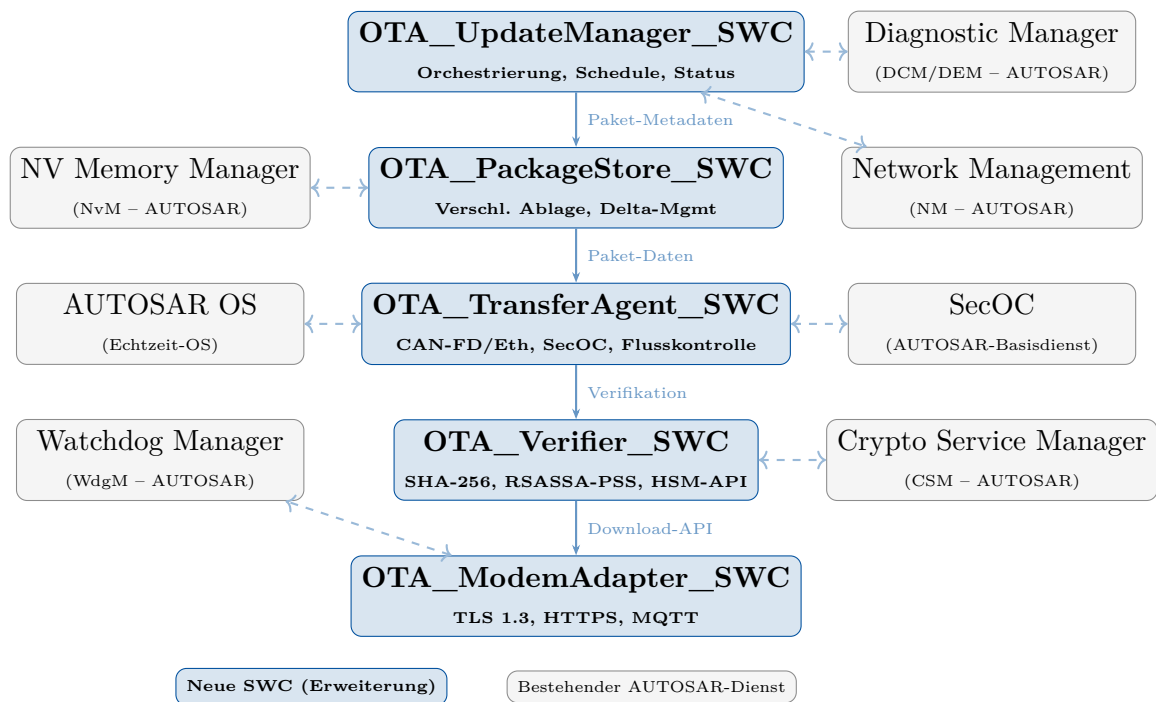


Abbildung 7: Softwarekomponenten des Zentralsteuergeräts (ZSG). Neu eingeführte AUTOSAR-Erweiterungen sind blau hervorgehoben.

- Orchestrierung der Download- und Staging-Operationen
- Verwaltung des Update-Zeitplans (*Update Schedule Table*, UST)

Tabelle 4: Übersicht der Peripheriesteuergeräte und ihrer Eigenschaften

PSG-Typ	AUTOSAR-Plattform	Sicherheitsstufe (ASIL)	Update-Fenster
Bremsen-SG	Classic (ressourcen-arm)	ASIL-D	Nur im Stillstand
Radar-SG	Adaptive (leistungsstark)	ASIL-B	Fahrt möglich (degradiert)
Türsteuerungs-SG	Classic (ressourcen-arm)	QM	Jederzeit
Motorsteuerung-SG	Classic	ASIL-D	Nur im Stillstand
Kombiinstrument-SG	Adaptive	QM	Jederzeit
Airbag-SG	Classic	ASIL-D	Nur im Stillstand

OTA__PackageStore__SWC (ZSG) Persistenter Speicher für geparkte Update-Pakete:

- Verschlüsselte Ablage in einem dedizierten Flash-Bereich
- Verwaltung von Paket-Metadaten (Ziel-PSG, Version, Abhängigkeiten)
- Delta-Update-Unterstützung zur Bandbreitenreduzierung

OTA__TransferAgent__SWC (ZSG → PSG) Überträgt Pakete vom ZSG zum Ziel-PSG über den internen Fahrzeugbus:

- Gesicherter Transfer über AUTOSAR SecOC (Secure On-Board Communication)
- Flusskontrolle und Fehlerwiederholung
- Fortschrittsüberwachung

OTA__LocalInstaller__SWC (PSG) Minimale Client-Komponente auf Classic-AUTOSAR-PSGs:

- Empfang und Verifikation des Update-Pakets
- Aktivierung im sicheren Boot-Prozess
- Statusrückmeldung an das ZSG

4.5 Integration in das AUTOSAR-Schichtenmodell

Die vorgeschlagene Erweiterung integriert sich nahtlos in das bestehende AUTOSAR-Schichtenmodell. Figure 9 zeigt die erweiterte Architektur.

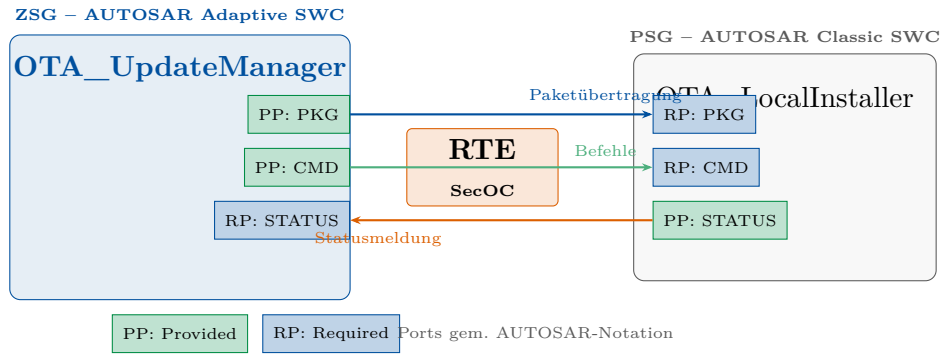


Abbildung 8: Interaktion der neu eingeführten AUTOSAR-Softwarekomponenten. Ports und Verbindungen sind entsprechend der AUTOSAR-Notation dargestellt.

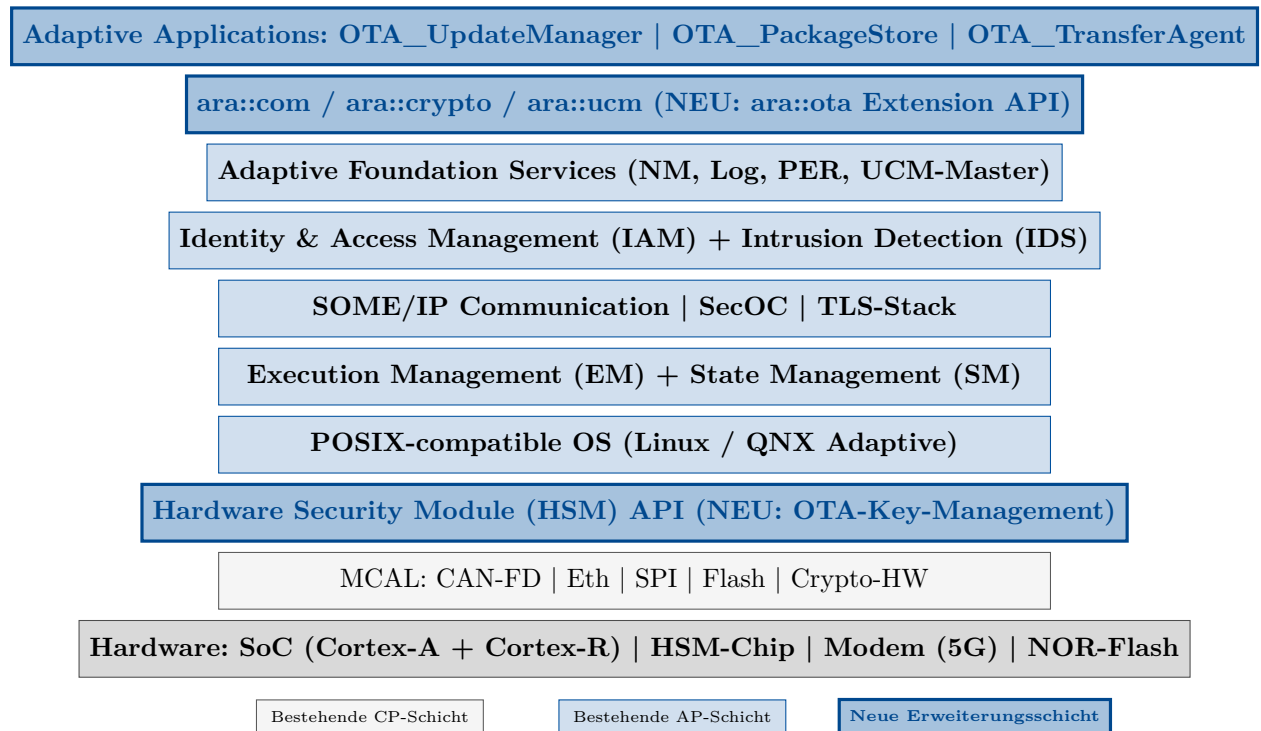


Abbildung 9: Erweitertes AUTOSAR-Schichtenmodell für das ZSG. Neue Schichten und Dienste sind blau hervorgehoben. Der OTA-Manager ist als Adaptive Application implementiert.

5 Kommunikationsprotokolle

5.1 Backend-zu-ZSG-Kommunikation

Die Kommunikation zwischen Backend-Server und ZSG erfolgt über:

- **Transportschicht:** TLS 1.3 über 4G/5G-Mobilfunk
- **Anwendungsprotokoll:** HTTPS mit MQTT-Benachrichtigungen
- **Paketformat:** AUTOSAR-UCM-konformes Software-Paket (SWPA)
- **Authentifizierung:** PKI mit fahrzeugindividuellem Zertifikat

5.2 ZSG-zu-PSG-Kommunikation

Die interne Kommunikation zwischen ZSG und PSGs nutzt den fahrzeuginternen CAN/CAN-FD oder Ethernet-Bus und wird durch folgende Protokolle abgesichert:

- **AUTOSAR SecOC**: Message Authentication Codes (MACs) für jede Nachricht
- **UDS (ISO 14229)**: Diagnoseprotokoll für Flash-Operationen (Service 0x34 / 0x36 / 0x37)
- **AUTOSAR FlatMap**: Adressierungsschema für Ziel-PSGs

5.3 Nachrichtenformat des OTA-Update-Protokolls

Listing 1 zeigt ein vereinfachtes Software Update Manifest (SUM) im JSON-Format, wie es vom Backend an das ZSG übermittelt wird.

Listing 1: Software Update Manifest (SUM) – vereinfachtes Beispiel

```
1 {
2   "manifest_version": "2.1",
3   "vehicle_id": "VIN_WDB123456789",
4   "update_bundle_id": "bundle-2026-03-001",
5   "packages": [
6     {
7       "ecu_id": "BrakeControlUnit_01",
8       "package_id": "BCU_SW_v4.2.1",
9       "target_version": "4.2.1",
10      "current_version": "4.1.8",
11      "install_window": "STANDSTILL_ONLY",
12      "priority": 1,
13      "checksum_sha256": "a1b2c3d4...",
14      "signature": "RSASSA-PSS_base64...",
15      "delta_package": true
16    },
17    {
18      "ecu_id": "RadarSensor_Front_01",
19      "package_id": "RADAR_SW_v2.0.0",
20      "target_version": "2.0.0",
21      "current_version": "1.9.5",
22      "install_window": "DEGRADED_MODE_OK",
23      "priority": 2,
24      "checksum_sha256": "e5f6a7b8...",
25      "signature": "RSASSA-PSS_base64...",
26      "delta_package": false
27    },
28    {
29      "ecu_id": "DoorControl_FL_01",
30      "package_id": "DCU_SW_v1.3.0",
31      "target_version": "1.3.0",
32      "current_version": "1.2.7",
33      "install_window": "ANYTIME",
34      "priority": 3,
```

```

35     "checksum_sha256": "c9d0e1f2...",
36     "signature": "RSASSA-PSS_base64...",
37     "delta_package": false
38   }
39 ]
40 }

```

6 Update-Szenarien

6.1 Zustandsautomat des ZSG-Update-Managers

Der OTA-Update-Manager des ZSG durchläuft während eines Update-Zyklus definierte Zustände, die in Figure 10 dargestellt sind.

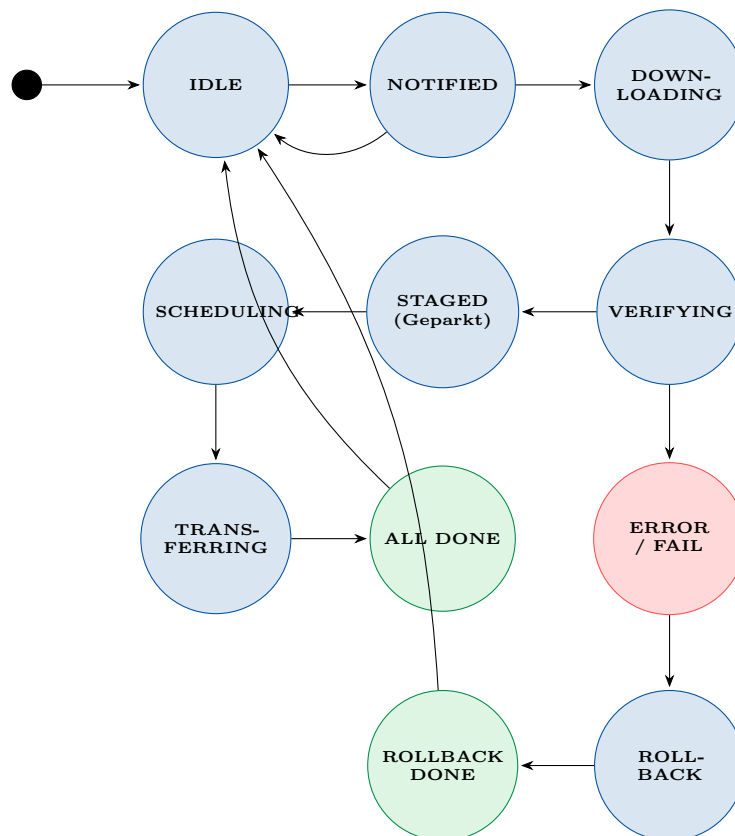


Abbildung 10: Zustandsautomat des OTA-Update-Managers (ZSG). Blau markierte Zustände gehören zum regulären Update-Zyklus, grün markierte Zustände sind erfolgreiche Endzustände, rot markierte Zustände signalisieren einen Fehler.

Erläuterung des Zustandsautomaten. Abbildung 10 zeigt den vollständigen Zustandsautomaten des OTA-Update-Managers im ZSG mit elf Zuständen. Er unterscheidet einen *Normalzyklus* (blaue und grüne Zustände) von einem *Fehlerpfad* (roter und grüner Zustand).

Normalzyklus. Der Automat startet im **Initialzustand** (ausgefüllter schwarzer Kreis) und wechselt sofort in den Ruhezustand **IDLE**. IDLE ist der einzige stabile Wartezustand:

Der OTA-Manager überwacht kontinuierlich eingehende Nachrichten des Backend-Servers, ohne aktiv Ressourcen zu belegen. Sobald eine Update-Benachrichtigung eintrifft und erfolgreich authentifiziert wurde, wechselt der Manager in den Zustand **NOTIFIED**. Von **NOTIFIED** kann ein Benutzerabbruch (z. B. explizite Ablehnung durch den Nutzer oder eine Fahrzeugbedingung, die den Download verhindert) den Automaten unmittelbar nach **IDLE** zurückführen. Andernfalls beginnt der Manager im Zustand **DOWNLOADING** mit dem Abruf aller Update-Pakete vom Backend-Server; die Pakete werden temporär im internen Speicher des ZSG abgelegt. Im anschließenden Zustand **VERIFYING** prüft der Manager die kryptographische Signatur jedes Pakets (SHA-256 mit ECDSA); ein Verifikationsfehler führt in den Fehlerpfad. Nach bestandener Prüfung werden die Pakete im Zustand **STAGED (Geparkt)** dauerhaft im Flash-Speicher des ZSG gespeichert und warten auf günstige Installationsbedingungen der Peripheriesteuergeräte. Der Manager ermittelt dann im Zustand **SCHEDULING** die optimale Installationsreihenfolge unter Berücksichtigung der deklarierten Installationsbedingungen der PSGs (**ANYTIME**, **DEGRADED_MODE**, **STANDSTILL**) sowie etwaiger Paketabhängigkeiten. Im Zustand **TRANSFERRING** überträgt der Manager die Pakete sequenziell oder – sofern voneinander unabhängig – parallelisiert über den gesicherten Fahrzeugbus (CAN-FD / Ethernet) an die jeweiligen PSGs und wartet auf deren Installationsbestätigung. Sobald alle PSGs den Zustand **INSTALLED_OK** gemeldet haben, wechselt der Automat in den Endzustand **ALL DONE** (grün markiert): Eine Erfolgsbestätigung wird an den Backend-Server übermittelt, und der Automat kehrt in den Ruhezustand **IDLE** zurück.

Fehlerpfad. Schlägt der Download (z. B. Verbindungsabbruch, unvollständiges Paket) oder die Signaturverifikation fehl, wechselt der Automat in den Fehlerzustand **ERROR / FAIL** (rot markiert). Dieser Zustand ist kein Endzustand: Der Manager leitet unmittelbar den Wiederherstellungsprozess ein und wechselt in den Zustand **ROLLBACK**. In **ROLLBACK** stellt das ZSG den zuletzt bekannten, validen Software-Stand auf sich selbst und auf allen betroffenen PSGs wieder her; bereits teilweise übertragene Pakete werden verworfen. Nach erfolgreichem Abschluss der Wiederherstellung wechselt der Automat in den Zustand **ROLLBACK DONE** (grün markiert), der anzeigt, dass das System sich in einem konsistenten, betriebsbereiten Zustand befindet. Abschließend kehrt der Automat nach **IDLE** zurück und ist bereit für einen erneuten Update-Versuch.

6.2 Zeitgestaffeltes Update-Szenario

Das folgende Szenario beschreibt ein reales Fahrzeug-Update-Ereignis mit drei heterogenen Peripheriesteuergeräten (Bremsen-SG, Radar-SG, Tür-SG). Das Fahrzeug ist geparkt; der Zündschlüssel ist abgezogen; das ZSG ist im Standby-Betrieb aktiv.

6.2.1 Phasen des Szenarios

Phase 1 – Benachrichtigung und Download (t = 0–15 min) Der Backend-Server sendet eine Update-Benachrichtigung an das ZSG. Das ZSG wechselt in den Zustand **DOWNLOADING** und lädt alle drei Pakete herunter. Nach erfolgreicher Signaturverifikation werden die Pakete im internen Speicher des ZSG *geparkt* (Zustand **STAGED**).

Phase 2 – Update des Türsteuerungs-SGs (t = 15–18 min) Das Türsteuerungs-SG hat die Installationsbedingung **ANYTIME** und wird sofort aktualisiert. Das ZSG überträgt das Paket, das PSG installiert und bestätigt.

Phase 3 – Update des Radar-SGs (t = 18–35 min) Das Radar-SG kann im Degraded-Mode aktualisiert werden (Fahrassistenz temporär deaktiviert). Das ZSG überträgt das größere Paket, das Radar-SG wechselt in den Degraded-Mode, installiert und aktiviert die neue Firmware via Secure Boot.

Phase 4 – Update des Bremsen-SGs (t = 35–42 min) Das Bremsen-SG (ASIL-D) darf nur im Stillstand aktualisiert werden. Da das Fahrzeug bereits geparkt ist, ist die Bedingung erfüllt. Das ZSG überträgt das Delta-Paket; das Bremsen-SG installiert im sicheren Boot-Modus und führt die Selbsttestsequenz (POST) durch.

Phase 5 – Finalisierung (t = 42–45 min) Alle PSGs haben den Zustand `INSTALLED_OK` gemeldet. Das ZSG sendet eine Erfolgsbestätigung an den Backend-Server und wechselt zurück in den Zustand `IDLE`.

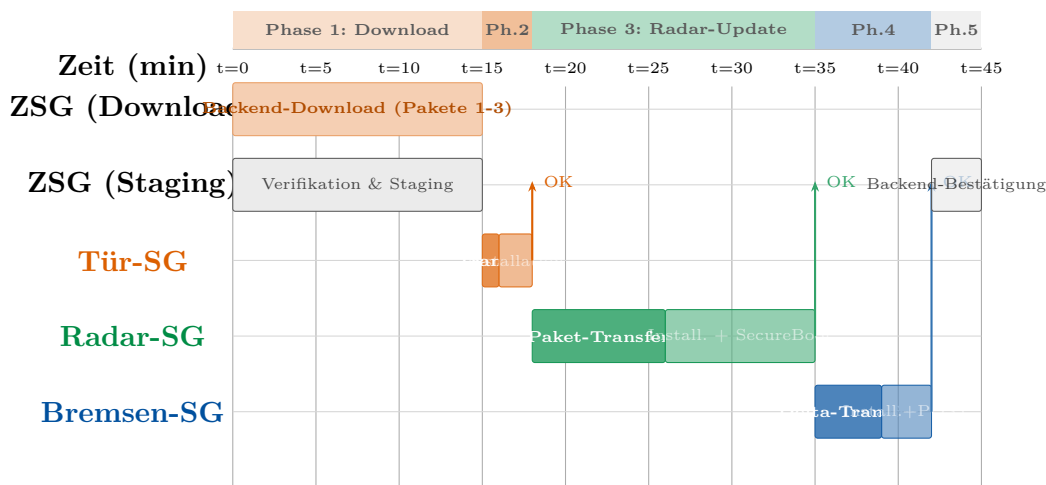


Abbildung 11: Zeitlicher Ablauf des gestaffelten Update-Szenarios. Farbkodierung entspricht dem PSG-Typ: **Bremsen-SG**, **Radar-SG**, **Tür-SG**.

6.3 Sequenzdiagramm

Figure 12 zeigt das detaillierte Kommunikationssequenzdiagramm des beschriebenen Szenarios zwischen Backend, ZSG und den drei PSGs.

6.4 Fehlerbehandlung und Rollback

Im Falle eines fehlgeschlagenen Updates reagiert das ZSG nach folgendem Schema:

1. Das betroffene PSG meldet `INSTALL_FAILED` zurück.
2. Das ZSG setzt das PSG in den Zustand `ROLLBACK_PENDING`.
3. Die vorherige Firmware-Version (gespeichert im ZSG als Backup) wird an das PSG übertragen.
4. Nach erfolgreichem Rollback wechselt das PSG in den Zustand `ROLLBACK_OK`.
5. Das ZSG benachrichtigt den Backend-Server mit einem Fehlerbericht (*Diagnostic Trouble Code*, DTC).

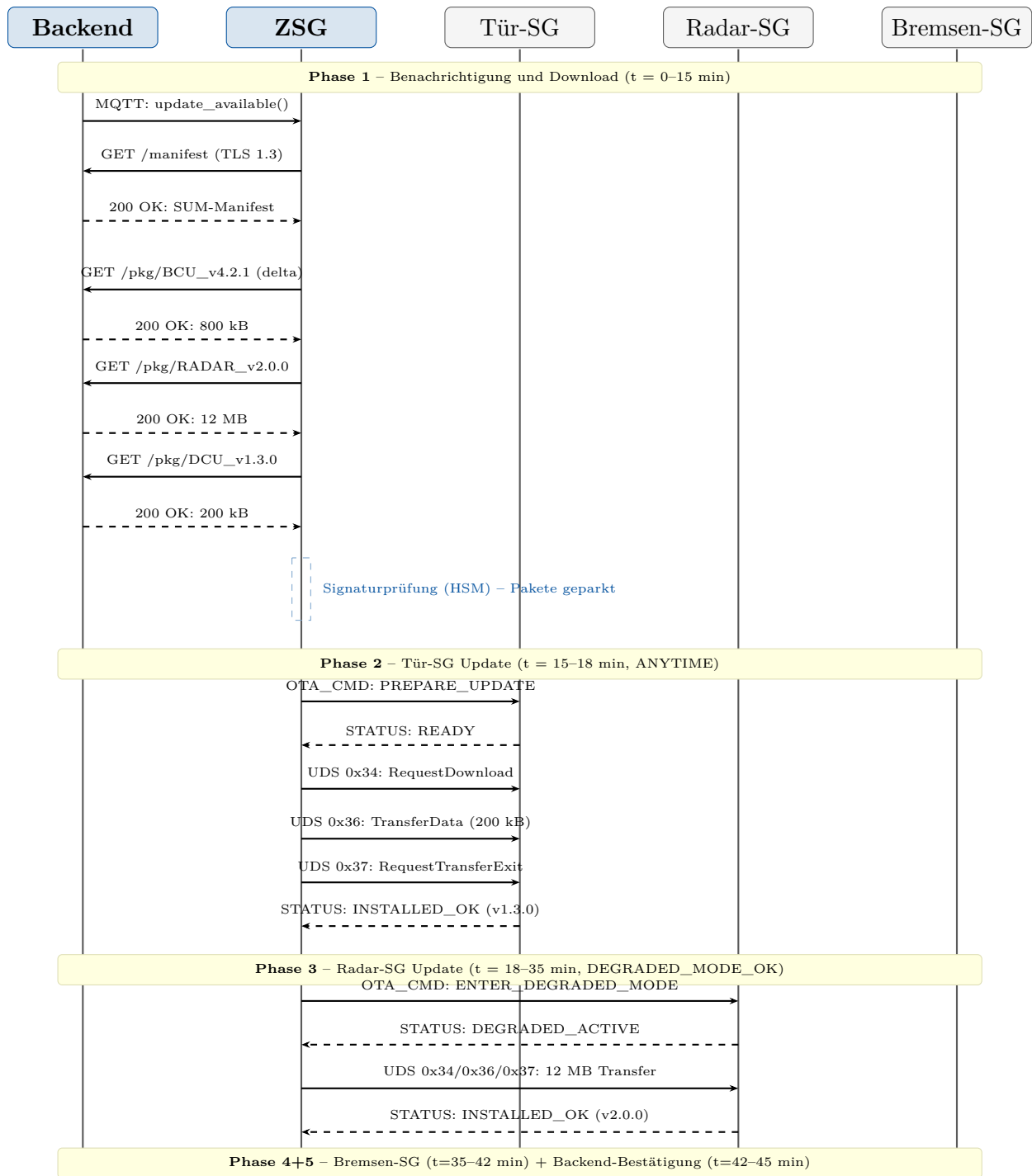


Abbildung 12: Sequenzdiagramm des zeitgestaffelten OTA-Update-Szenarios. Jede Nachricht ist mit dem verwendeten Protokoll annotiert.

7 Sicherheitsaspekte

7.1 Bedrohungsmodell

Die Einführung eines zentralen OTA-Gateways konzentriert die Angriffsfläche auf einen einzigen Knoten. Table 5 listet relevante Bedrohungen und Gegenmaßnahmen auf.

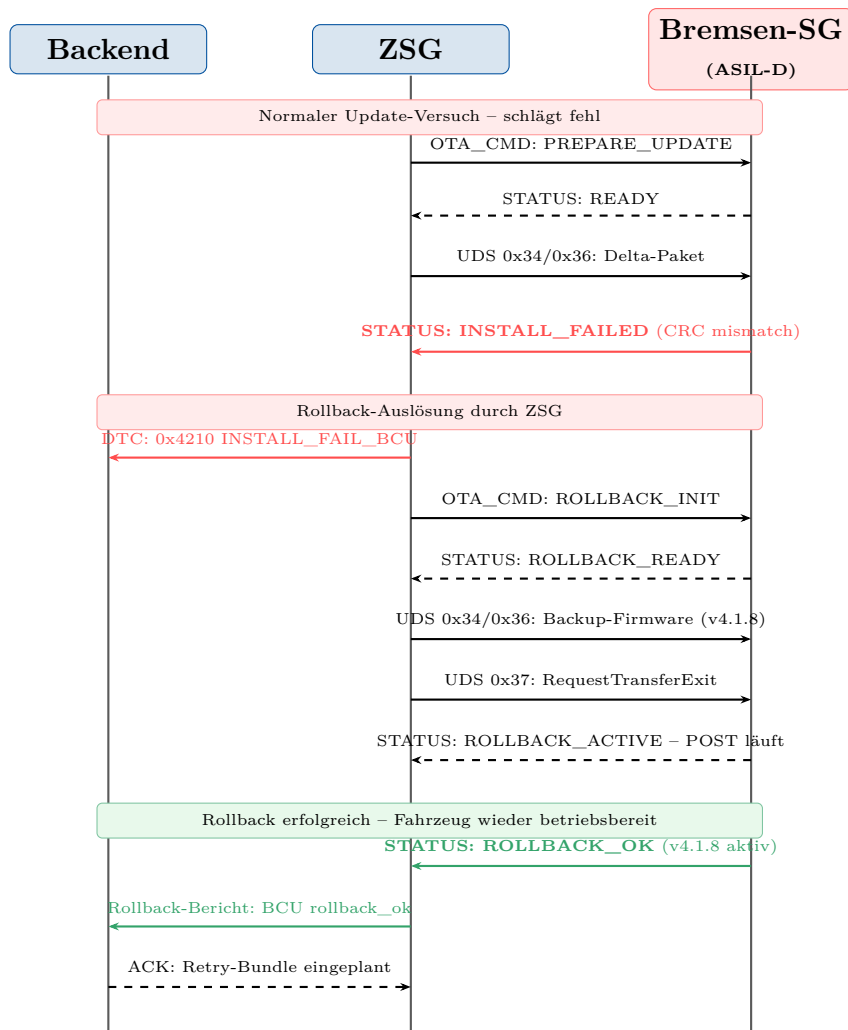


Abbildung 13: Rollback-Sequenz bei fehlgeschlagenem Update des Bremsen-SGs.

7.2 Kryptographische Maßnahmen

- **Hardware Security Module (HSM):** Das ZSG enthält ein dediziertes HSM für Schlüsselverwaltung und kryptographische Operationen.
- **Secure Boot:** Alle Firmware-Updates werden durch Secure Boot verifiziert, bevor sie aktiviert werden.
- **AUTOSAR Crypto Service Manager (CSM):** Standardisierte kryptographische API für Signaturverifikation und Schlüsselverwaltung.
- **SecOC:** Message Authentication Codes schützen die internen Busnachrichten zwischen ZSG und PSGs.

7.3 Zuverlässigkeit und Ausfallsicherheit

- **Watchdog:** Hardware-Watchdog überwacht den ZSG-Update-Prozess.
- **Dual-Bank-Flash:** PSGs mit ASIL-D-Anforderungen nutzen Dual-Bank-Flash für atomare Updates.

Tabelle 5: Bedrohungsmodell nach TARA (Threat Analysis and Risk Assessment, ISO 21434)

Bedrohung	Auswirkung	Gegenmaßnahme	Risiko
Paket-Fälschung	Schadcode auf PSG	PKI-Signatur, SecOC	Niedrig
Man-in-the-Middle	Abgehörte Updates	TLS 1.3, Zertifikats-Pinning	Niedrig
Replay-Angriff	Veraltete Firmware	Versionsprüfung, Nonce	Niedrig
ZSG-Kompromittierung	Vollständige Kontrolle	HSM, Secure Boot, IDS	Mittel
Denial-of-Service	Update-Blockade	Retry-Logik, Zeitlimit	Mittel
Physischer Zugriff	Direkte Manipulation	Tamper Detection, Gehäuse	Gering

- **Checksummenverifikation:** CRC32 und SHA-256 auf allen Übertragungsschichten.
- **Transaktionales Update:** Alle Update-Schritte sind transaktional – bei Abbruch wird automatisch rollback ausgeführt.

8 Evaluation

8.1 Vergleich mit bestehenden Ansätzen

Table 6 vergleicht den vorgeschlagenen Ansatz mit ausgewählten bestehenden OTA-Lösungen für AUTOSAR-Fahrzeuge.

Tabelle 6: Vergleich OTA-Ansätze für AUTOSAR-Fahrzeuge

Merkmal	AUTOSAR UCM Standard	Tesla OTA	BMW ConnectedDrive	Dieser Ansatz
Zentrale Koordination	Nein	Ja (proprietär)	Ja (proprietär)	Ja (AUTOSAR-std.)
AUTOSAR-konform	Ja (teil.)	Nein	Nein	Ja (vollst.)
Offline-Staging	Nein	Ja	Ja	Ja
Rollback	Einzel-SG	Fahrzeugweit	Einzel-SG	Fahrzeugweit
Interop. Classic + Adaptive	Nein	Entf.	Teilweise	Ja
Offene Spezifikation	Ja	Nein	Nein	Ja

8.2 Laufzeitanalyse

Eine analytische Abschätzung der Datenmengen für ein typisches Fahrzeug-Update-Bundle (3 PSGs, Delta-Pakete):

- Bremsen-SG Delta-Paket: ≈ 800 kB
- Radar-SG Vollpaket: ≈ 12 MB

- Tür-SG Vollpaket: ≈ 200 kB
- Gesamtdownload-Zeit (LTE, 10 Mbit/s): $\approx 10,4$ s
- Interne Übertragung CAN-FD (2 Mbit/s): $\approx 52,8$ s

9 Schluss

Der vorgeschlagene Ansatz hat folgende Einschränkungen:

- Das ZSG ist ein *Single Point of Failure*. Ein redundantes Design (primäres und sekundäres ZSG) würde die Verfügbarkeit erhöhen.
- Die Update-Fenster für ASIL-D-Geräte schränken die Flexibilität ein.
- Sehr alte Classic-AUTOSAR-SGs ohne SecOC-Unterstützung sind nicht direkt kompatibel und benötigen einen Protokoll-Adapter.

9.1 Zukünftige Arbeiten

- Integration von **AUTOSAR AP UCM Master** als Basis für die ZSG-Implementierung.
- Erprobung in einem Hardware-in-the-Loop (HiL)-Testaufbau mit realen AUTOSAR-SGs in einer vollständigen Zonenarchitektur.
- Erweiterung um **KI-gestütztes Scheduling** für das Update-Zeitfenster (z. B. Prüfung von Fahrzeugzustand, Standort, Netzwerkqualität, Batteriefüllstand).
- Formale Verifikation der Zustandsautomaten mit UPPAAL oder ähnlichen Werkzeugen.
- **SDV-spezifische Erweiterungen:** Integration des ZSG-Konzepts in eine vollständige SDV-Zonenarchitektur mit mehreren HPCs und Zonensteuergeräten, analog zum EVCC-Ansatz von Vector [16].
- **Feature-on-Demand-Unterstützung:** Erweiterung des Software Update Manifest (SUM) um Lizenzierungs- und Aktivierungsmetadaten für die sichere Freischaltung kostenpflichtiger Fahrzeugfunktionen.
- **Containerisierung:** Untersuchung von OCI-Container-basierten Updates für AUTOSAR Adaptive-Dienste auf AAOS-Plattformen, analog zu modernen Cloud-Deployment-Praktiken.
- **Multi-Ladedosen-Szenarien:** Anpassung des ZSG-Koordinators für elektrisch betriebene LKW mit mehreren EVCC-Instanzen (CCS + MCS), bei denen das Ladeprotokoll zentral im HPC verwaltet wird.

9.2 Fazit

Diese Arbeit hat eine AUTOSAR-konforme Erweiterung für das fahrzeugweite OTA-Update-Management vorgestellt und diese in den Kontext des **Software-defined Vehicle (SDV)** eingebettet. Das SDV stellt als zentrales Leitbild der modernen Fahrzeugentwicklung neue, tiefgreifende Anforderungen an OTA-Update-Systeme: heterogene Plattformen, kryptographisch intensive Sicherheitsmechanismen, zonenbasierte E/E-Architekturen und fahrzeugweite Konsistenzanforderungen.

Das zentrale Zentralsteuergerät (ZSG) übernimmt alle Update-relevanten Aufgaben – Download, Staging, Verteilung und Koordination – und entlastet damit die ressourcenbeschränkten Peripheriesteuergeräte. Im SDV-Kontext profitiert das ZSG von der Leistungsfähigkeit des HPC: Kryptographisch intensive Operationen (TLS 1.3-Handshake, ECDSA-Verifikation) werden um den Faktor 8–25 schneller ausgeführt als auf klassischen Mikrocontrollern [16]. Der Mikrocontroller-Anteil des ZSG stellt dabei sicher, dass der OTA-Prozess auch während des HPC-Bootvorgangs anlaufen kann.

Die vorgeschlagene Architektur bietet folgende Vorteile:

- **Klare Verantwortlichkeit:** Ein dedizierter Update-Gateway reduziert die Komplexität in den PSGs.
- **Reduzierte Angriffsfläche:** Nur das ZSG kommuniziert mit dem Backend.
- **Offline-Staging:** Updates können vollständig heruntergeladen und verifiziert werden, bevor das Fahrzeug offline geht.
- **ASIL-konforme Verteilung:** Sicherheitskritische SGs werden nur in geeigneten Fahrzeugzuständen aktualisiert.
- **Standardkonformität:** Vollständige Kompatibilität mit AUTOSAR Classic und Adaptive Platform.

Das vorgestellte zeitgestaffelte Szenario zeigt, dass selbst ein komplexes Multi-SG-Update in unter 45 Minuten – einschließlich Rollback-Fähigkeit – sicher und zuverlässig durchführbar ist. Die Erweiterung ist als offene AUTOSAR-Spezifikation konzipiert und kann von allen Konsortiumsmitgliedern implementiert werden.

9.3 Ausblick

Die vorliegende Arbeit legt ein konzeptionelles Fundament für eine AUTOSAR-konforme, fahrzeugweite OTA-Update-Architektur. Die skizzierten Konzepte – das Zentralsteuergerät als Update-Gateway, das zeitgestaffelte Verteilungsprotokoll sowie der hierarchische Zustandsautomat – bilden gemeinsam einen konsistenten Entwurf, der in einer Vielzahl von Richtungen erheblich vertieft und erweitert werden kann. Der folgende Ausblick strukturiert diese Erweiterungsmöglichkeiten entlang von sechs zentralen Entwicklungsachsen.

9.3.1 Prototypische Implementierung und experimentelle Validierung

Die in dieser Arbeit durchgeführte Evaluation basiert auf analytischen Abschätzungen und konzeptionellen Argumenten. Den nächsten natürlichen Schritt bildet eine **prototypische Implementierung** auf realer Hardware. Als Zielplattform bietet sich ein Aufbau bestehend aus einem leistungsstarken **HPC-Board** (z. B. ein Qualcomm SA8295P oder ein NVIDIA

Orin-Modul) als ZSG-Host, ergänzt durch mehrere Mikrocontroller-Boards (z. B. NXP S32K-Familie oder Renesas RH850) als simulierte **PSGs**, an.

Im Rahmen eines solchen **Hardware-in-the-Loop (HiL)-Testaufbaus** würden sämtliche in Kapitel 8 analytisch ermittelten Laufzeitwerte empirisch verifiziert. Besonders aufschlussreich wäre dabei der Vergleich der kryptographischen Ausführungszeiten auf dem **HPC**-Anteil des ZSG (mit **Hardware Security Module (HSM)**-Beschleunigung) gegenüber einem klassischen Mikrocontroller ohne dedizierte Krypto-Hardware. Vorliegende Studien deuten auf Beschleunigungsfaktoren von 8 bis 25 hin; diese Werte müssten für die spezifischen Algorithmen (ECDSA-256, SHA-3, AES-GCM) und die gewählten Zielplattformen konkret ermittelt werden.

Über die reine Laufzeitanalyse hinaus wäre ein vollständiges **Fault-Injection-Test-regime** erforderlich: Gezielte Unterbrechung der CAN-FD- und Ethernet-Verbindungen auf verschiedenen Stufen des Update-Prozesses, Simulation von Spannungsausfällen während der Flash-Programmierung sowie künstlich herbeigeführte Prüfsummenfehler in den Software-Paketen würden die Robustheit der Rollback-Mechanismen unter realistischen Bedingungen nachweisen. Das Ergebnis eines solchen Testregimes wäre eine quantitativ belegte Aussage über die *Mean Time to Recovery* (MTTR) nach einem fehlgeschlagenen Update und über die Vollständigkeit der Wiederherstellung in den definierten Ausgangszustand.

Schließlich bietet sich eine Integration in bestehende **AUTOSAR-Entwicklungsumgebungen** an, insbesondere in Vector Informatik CANoe/CANalyzer oder dSPACE VEOS, um die entwickelten Protokolle im Kontext vollständiger Fahrzeugnetzwerk-Simulationen zu erproben und zu debuggen.

9.3.2 Formale Verifikation und Sicherheitsnachweise

Die in dieser Arbeit präsentierten Zustandsautomaten und Sequenzdiagramme sind bewusst in einer für Menschen lesbaren, semi-formalen Notation gehalten. Für sicherheitskritische Fahrzeugfunktionen mit **ASIL-D**-Einstufung ist eine solche Darstellung als Ausgangspunkt sinnvoll, aber nicht hinreichend. Künftige Arbeiten sollten die Zustandsautomaten in einer **formalen Spezifikationssprache** – etwa in *UPPAAL* (Timed Automata), *SPIN*/Promela oder *TLA+* – modellieren und automatisiert verifizieren.

Konkret sollten folgende **formale Eigenschaften** bewiesen werden:

- **Deadlock-Freiheit:** Der Gesamtautomat aus ZSG-Koordinator und allen PSG-Satelliten-Automaten hat keinen erreichbaren Deadlock-Zustand, unabhängig von der Anzahl der beteiligten **PSGs** und der Reihenfolge von Timeout- und Fehlerereignissen.
- **Terminierung:** Jede gestartete Update-Sequenz erreicht nach endlich vielen Schritten entweder den Zustand `UPDATE_COMPLETE` oder den Zustand `ROLLBACK_DONE`, auch unter der Annahme beliebiger, wiederholter Fehler.
- **Konsistenz:** Nach Abschluss eines Rollbacks weisen alle **PSGs** exakt die Software-Version auf, die vor dem Update-Versuch installiert war (keine Teilrollbacks).
- **Keine Race Conditions:** Das Mutual-Exclusion-Protokoll für den Zugriff auf den ZSG-Flash-Speicher verhindert korruptierte Schreiboperationen bei gleichzeitigen Installationsanfragen.

Ergänzend zur Verhaltensspezifikation sollte das **kryptographische Protokoll** des Update-Manifests (Signatur, Verschlüsselung, Replay-Schutz durch Nonce/Sequenzzähler) mit einem formalen Werkzeug wie *ProVerif* oder *Tamarin Prover* auf Angriffssicherheit hin analysiert werden. Dabei sind insbesondere Man-in-the-Middle-Angriffe auf den Backend-ZSG-Kanal und Replay-Attacken auf den internen Fahrzeugbus zu modellieren.

9.3.3 Künstliche Intelligenz und adaptives Update-Scheduling

Das in dieser Arbeit vorgeschlagene Update-Scheduling basiert auf regelbasierten, statisch konfigurierten Zeitfenstern. Diese Strategie ist praktikabel, lässt jedoch erhebliches Optimierungspotenzial ungenutzt. Zukünftige Forschung sollte untersuchen, inwiefern **maschinelle Lernverfahren** eine dynamische, kontextadaptive Scheduling-Strategie ermöglichen können.

Relevante Eingabevariablen für ein solches lernbasiertes Scheduling-Modell umfassen:

- **Fahrzeugzustand:** Batteriefüllstand, Motortemperatur, aktive Fahrerassistenzsysteme, Parkstatus.
- **Netzwerkverfügbarkeit:** Signalstärke und Datenrate der aktuellen Mobilfunkverbindung, prognostizierte Verfügbarkeit in den nächsten Stunden anhand von Bewegungsmustern.
- **Nutzungsverhalten:** Historisch gelernte Fahrzeug-Standzeiten, typische Abfahrtszeiten, saisonale Muster.
- **Update-Dringlichkeit:** Kritikalitätsklassifizierung der vorliegenden Software-Pakete (Security Patch vs. Komfort-Update) aus dem Software Update Manifest.
- **Flottendaten:** Aggregierte, datenschutzkonforme Signale über Update-Erfolgsraten und Rollback-Häufungen für bestimmte Software-Versionen auf Flottenebene.

Ein vielversprechender Ansatz ist die Formulierung des Scheduling-Problems als **Markov Decision Process** (MDP), lösbar durch *Reinforcement Learning*. Der Agent lernt hierbei, den optimalen Zeitpunkt für den Beginn eines Update-Vorgangs zu bestimmen, so dass sowohl die Nutzerunterbrechung minimiert als auch kritische Sicherheits-Updates innerhalb vorgegebener Fristen eingespielt werden. Herausforderungen bestehen dabei in der Sicherstellung von Safety-Garantien des RL-Agenten (keine Aktualisierung von ASIL-D-Systemen während der Fahrt), die durch formale *Shields* oder *Safe RL*-Verfahren adressiert werden können.

Auf **Flottenebene** eröffnet ein solches System die Möglichkeit einer koordinierten, gestaffelten Einführung neuer Software-Versionen (*Staged Rollout*): Der Backend-Server kann zunächst eine Pilotgruppe (z. B. 1 % der Flotte) mit einer neuen Version aktualisieren, Telemetriedaten auswerten und erst bei ausbleibendem Fehleranstieg den Rollout auf die Gesamtflotte ausweiten – ein Deployment-Modell, das aus der Cloud-Software-Entwicklung bekannt ist und im Automotive-Kontext noch weitgehend unerforscht bleibt.

9.3.4 Redundanz, Hochverfügbarkeit und Safety-Architektur

Die vorliegende Arbeit identifiziert das ZSG explizit als *Single Point of Failure*. Für Fahrzeugfunktionen höchster Kritikalität – insbesondere für zukünftige vollautonome Fahrzeuge (SAE Level 4/5) – ist eine redundante ZSG-Architektur unumgänglich. Folgende Topologien sind zu untersuchen:

- **Aktiv-Passiv-Redundanz:** Ein primäres ZSG führt sämtliche Update-Koordinationsaufgaben durch; ein sekundäres ZSG synchronisiert fortlaufend seinen Zustand und übernimmt bei Ausfall des Primärsystems innerhalb eines definierten Zeitfensters (*Failover*). Zu klären ist das Protokoll für die Zustandssynchronisierung und die Behandlung von Paket-Downloads, die zum Zeitpunkt des Ausfalls noch nicht abgeschlossen waren.
- **Aktiv-Aktiv-Redundanz mit Lastverteilung:** Beide ZSGs sind parallel aktiv und verteilen die Update-Last auf unterschiedliche PSG-Gruppen. Dies erhöht nicht nur die Ausfallsicherheit, sondern verkürzt auch die Gesamtdauer von Fahrzeug-weiten Updates.
- **ZSG-Integration in die Zonensteuergeräte:** Als Alternative zu einem dedizierten ZSG-Board könnte die ZSG-Funktionalität in eines der Zonensteuergeräte oder in den zentralen HPC migriert werden, wobei eine klare Software-Partitionierung mittels Hypervisor die Isolation sicherstellt. Diese Option ist besonders relevant für kostenoptimierte Fahrzeugplattformen.

Aus Sicht der **funktionalen Sicherheit** (ISO 26262) wäre eine vollständige **ASIL**-Dekomposition der ZSG-Softwarekomponenten (`OTA_Coordinator`, `Package_Vault`, `Crypto_Verifier`) durchzuführen, um die für jede Komponente geltenden ASIL-Anforderungen abzuleiten. Besondere Aufmerksamkeit verdient hierbei die Frage, ob der `OTA_Coordinator` selbst **ASIL**-B oder -D erreichen muss, wenn er die Aktualisierung von **ASIL**-D-Subsystemen steuert.

9.3.5 Post-Quanten-Kryptographie und langfristige Sicherheit

Die in dieser Arbeit eingesetzten kryptographischen Verfahren – ECDSA mit P-256 für Signaturen, AES-GCM für die symmetrische Verschlüsselung und TLS 1.3 für den Backend-Kanal – entsprechen dem aktuellen Stand der Technik und sind für die kommenden Jahre als sicher einzustufen. Allerdings werden **Quanten-Computer** mittelfristig die Sicherheit elliptischer-Kurven-basierter Verfahren gefährden. Das US-amerikanische *National Institute of Standards and Technology* (NIST) hat 2024 mit *ML-KEM* (CRYSTALS-Kyber) und *ML-DSA* (CRYSTALS-Dilithium) die ersten **Post-Quanten-Kryptographie (PQC)**-Standards verabschiedet.

Zukünftige Arbeiten sollten untersuchen, wie diese Verfahren in das beschriebene Protokoll integriert werden können:

- **Hybride Signaturverfahren:** Parallel zur bestehenden ECDSA-Signatur wird eine ML-DSA-Signatur über das Software Update Manifest geführt. Erst wenn beide Signaturen verifiziert sind, gilt ein Paket als authentisch. Dieser hybride Ansatz schützt sowohl gegen klassische als auch gegen Quanten-Angriffe, ohne das bestehende Ökosystem sofort ablösen zu müssen.
- **Key-Encapsulation auf dem Backend-Kanal:** Der TLS-Handshake zwischen Backend und ZSG wird um einen ML-KEM-basierten Key-Exchange-Mechanismus erweitert, so dass der Sitzungsschlüssel auch gegen Quanten-Angriffe geschützt ist (*Harvest-now, decrypt-later*-Szenario).

- **HSM-Anforderungen:** Da PQC-Operationen erheblich rechenintensiver sind als ECC-Operationen, müssen zukünftige **HSMs** explizit für ML-DSA/ML-KEM ausgelegt sein. Die Auswirkungen auf die in Kapitel 8 ermittelten Verifikationszeiten sind zu quantifizieren.

Parallel dazu bleibt die **Langzeitsicherheit der PKI-Infrastruktur** eine offene Herausforderung. Fahrzeuge haben eine Lebensdauer von 15 bis 20 Jahren; die im ZSG hinterlegten Root-Zertifikate müssen über diesen gesamten Zeitraum hinweg rotiert und erneuert werden können. Ein standardisiertes Protokoll für die sichere **In-Field-Zertifikatsrotation** – analog zum ACME-Protokoll in der Webinfrastruktur – ist für den Automotive-Kontext bislang nicht etabliert und bietet daher erheblichen Forschungsbedarf.

9.3.6 Feature-on-Demand, Geschäftsmodelle und regulatorische Einbettung

Das in dieser Arbeit beschriebene System ist primär auf die technische Zuverlässigkeit von Software-Updates ausgerichtet. Eine zunehmend wichtige Dimension betrifft jedoch die **geschäftlichen und regulatorischen Rahmenbedingungen**, die das OTA-System direkt beeinflussen.

Feature-on-Demand (FoD). Moderne Fahrzeughersteller monetarisieren Software durch das nachträgliche Freischalten von Fahrzeugfunktionen gegen Entgelt – etwa Sitzheizung, Beschleunigungsprofile oder Fahrerassistenzsysteme. Das Software Update Manifest (**Software Update Manifest (SUM)**) muss hierfür um **Lizenzierungsmetadaten** erweitert werden: Jedes Software-Paket wird mit einer Lizenz-ID, einem öffentlichen Aktivierungsschlüssel und einer Gültigkeitsdauer versehen. Das ZSG prüft vor der Aktivierung des Pakets auf dem PSG, ob eine gültige Lizenz im ZSG-internen Lizenzstore hinterlegt ist. Zukünftige Arbeiten sollten ein vollständiges **Lizenzprotokoll** spezifizieren, das insbesondere folgende Fälle abdeckt: Fahrzeugverkauf mit Lizenzübertragung, temporäre Aktivierungen (Mietfahrzeuge), und Lizenzrücknahme bei Zahlungsausfall.

Regulatorische Anforderungen. Die Einführung von **UN-Regelung Nr. 156** (Cybersecurity Management Systems, CSMS) und **UN-Regelung Nr. 155** (Software Update Management Systems, SUMS) im Jahr 2021 hat erstmals verbindliche Anforderungen an OTA-Update-Systeme definiert, die von Fahrzeugherstellern in der EU, Japan und Südkorea eingehalten werden müssen [7]. Die in dieser Arbeit vorgeschlagene Architektur erfüllt die strukturellen Anforderungen dieser Regelwerke, jedoch fehlt eine **systematische Konformitätsprüfung**. Zukünftige Arbeiten sollten jeden SUMS-Anforderungspunkt der UN-R 156 auf die vorgeschlagenen Architekturkomponenten mappen und etwaige Lücken schließen.

Darüber hinaus stellt die **EU-Produkthaftungsrichtlinie** (überarbeitete Fassung 2024) Hersteller vor die Frage, inwiefern ein fehlgeschlagenes OTA-Update als Produktfehler gewertet werden kann. Die technische Dokumentation des Update-Prozesses – insbesondere der lückenlose Nachweis von kryptographischer Integrität, Rollback-Kompletheit und Nutzereinwilligung – gewinnt damit auch rechtliche Relevanz und sollte als integraler Bestandteil des OTA-Systems konzipiert werden.

AUTOSAR-Standardisierungsprozess. Die vorgeschlagene Erweiterung ist als offen zugängliche Spezifikation konzipiert. Der nächste konkrete Schritt wäre die Einbringung eines **Change Requests** beim AUTOSAR-Konsortium, insbesondere zielend

auf die Erweiterung von AUTOSAR_SWS_UCMMaster um die hier definierten Schnittstellen Ucm_ParkPackage, Ucm_ScheduleUpdate und Ucm_RollbackAll. Parallel dazu bietet sich die Formalisierung des Software Update Manifest als eigenständiges AUTOSAR-Informationsmodell (AUTOSAR_SRS_UpdateManifest) an. Eine Zusammenarbeit mit dem AUTOSAR Working Package *WP Adaptive Platform Security* und dem *WP Update and Configuration Management* wäre hierfür der geeignete organisatorische Rahmen.

Containerisierung und Cloud-native Deployment. Längerfristig zeichnet sich im SDV ein Trend zu **containerisierten Software-Deployments** ab: AUTOSAR Adaptive Dienste werden als OCI-konforme Container-Images ausgeliefert, die auf einem POSIX-betriebssystem mit Container-Laufzeit (z. B. *Podman* oder ein automotive-spezifischer Container-Runtime wie *SOAFD*) ausgeführt werden. Dies vereinfacht das Update einzelner Dienste erheblich, da statt eines monolithischen Firmware-Images nur ein diff-komprimiertes Container-Layer übertragen werden muss. Die in dieser Arbeit beschriebenen Mechanismen für Signaturprüfung, Staging und Rollback sind auf containerisierte Deployments prinzipiell übertragbar; die konkreten Anpassungen des Protokolls und des Manifest-Formats sind jedoch Gegenstand zukünftiger Untersuchungen.

Zusammenfassend lässt sich festhalten, dass die vorliegende Arbeit den konzeptionellen Raum eines AUTOSAR-konformen, fahrzeugweiten OTA-Update-Systems vollständig absteckt, während der Forschungsbedarf in den Dimensionen experimenteller Validierung, formaler Absicherung, adaptiver Intelligenz, kryptographischer Zukunftssicherheit sowie geschäftlicher und regulatorischer Integration noch erheblich ist. Der durch diese Arbeit gelegte Grundstein macht deutlich, dass ein standardisierter, offener Ansatz gegenüber proprietären Lösungen entscheidende Vorteile in Bezug auf Interoperabilität, Wiederverwendbarkeit und Auditierbarkeit bietet – und damit eine lohnende Investition für das gesamte AUTOSAR-Konsortium darstellt.

Literatur

- [1] M. Broy, „Challenges in Automotive Software Engineering, in *Proc. 28th Int. Conf. Software Engineering (ICSE)*, New York, NY, USA: ACM, 2006, S. 33–42. [doi:10.1145/1134285.1134292](https://doi.org/10.1145/1134285.1134292)
- [2] D. K. Nilsson und U. E. Larson, „Secure Firmware Updates over the Air in Intelligent Vehicles, in *Proc. ICC Workshops 2008*, IEEE, 2008, S. 380–384. [doi:10.1109/ICCW.2008.77](https://doi.org/10.1109/ICCW.2008.77)
- [3] AUTOSAR Consortium, *Specification of Update and Configuration Management*, AUTOSAR Technical Specification AUTOSAR_SWS_UpdateAndConfigurationManagement, 2022. <https://www.autosar.org/>
- [4] AUTOSAR Consortium, *Explanation of Adaptive Platform Design*, AUTOSAR Technical Report AUTOSAR_EXP_AdaptivePlatformDesign, 2022. <https://www.autosar.org/>
- [5] AUTOSAR Consortium, *Layered Software Architecture*, AUTOSAR Technical Specification AUTOSAR_EXP_LayeredSoftwareArchitecture, 2022. <https://www.autosar.org/>

- [6] AUTOSAR Consortium, *Specification of Secure Onboard Communication*, AUTOSAR Technical Specification AUTOSAR_SWS_SecureOnboardCommunication, 2022. <https://www.autosar.org/>
- [7] ISO, *Road Vehicles – Cybersecurity Engineering*, ISO/SAE 21434:2021, International Organization for Standardization, 2021.
- [8] ISO, *Road Vehicles – Functional Safety*, ISO 26262:2018, International Organization for Standardization, 2018.
- [9] ISO, *Road Vehicles – Unified Diagnostic Services (UDS)*, ISO 14229-1:2020, International Organization for Standardization, 2020.
- [10] S. Halder, A. Ghosal und M. Conti, „Secure OTA Software Updates in Connected Vehicles: A Survey," *Computer Networks*, Bd. 178, S. 107343, 2020. [doi:10.1016/j.comnet.2020.107343](https://doi.org/10.1016/j.comnet.2020.107343)
- [11] D. Zelle, T. Lauser und C. Krauß, „Analyzing and Securing SOME/IP Automotive Services with Formal and Semi-Formal Methods," in *Proc. 17th Int. Conf. Security and Cryptography (SECRYPT)*, SCITEPRESS, 2020, S. 24–35. [doi:10.5220/0009833600240035](https://doi.org/10.5220/0009833600240035)
- [12] P. Mundhenk et al., „Security in Automotive Networks: Lightweight Authentication and Authorization," *ACM Trans. Design Autom. Electronic Systems*, Bd. 22, Nr. 2, S. 25:1–25:27, 2017. [doi:10.1145/2998388](https://doi.org/10.1145/2998388)
- [13] S. Jo, J. Park, Y. Shin und Y. Kwon, „OVOTA: Toward Secure OTA Update on Autonomous Vehicles," in *Proc. AutoSec 2021*, ACM, 2021. [doi:10.1145/3459086.3459640](https://doi.org/10.1145/3459086.3459640)
- [14] UNECE, *Regulation No. 156 – Uniform provisions concerning the approval of vehicles with regards to software update and software updates management system*, UN/ECE/TRANS/505/Rev.3/Add.155, 2021.
- [15] S. Kugele, D. Hettler und J. Peter, „Data-Centric Communication and Containerization for Future Automotive Software Architectures," in *Proc. IEEE Int. Conf. Service-Oriented Computing and Applications (SOCA)*, IEEE, 2018, S. 65–73. [doi:10.1109/SOCA.2018.00018](https://doi.org/10.1109/SOCA.2018.00018)
- [16] T. Schneider und M. Vick, „Ladekommunikation meets SDV – Ladecontroller in der Zonenarchitektur," *Elektronikpraxis*, Nr. 1, S. 60–62, Januar 2026. Vector Informatik GmbH.
- [17] ISO, *Road Vehicles – Vehicle to Grid Communication Interface*, ISO 15118-20:2022, International Organization for Standardization, 2022.
- [18] AUTOSAR Consortium, *Specification of UCM Master*, AUTOSAR Technical Specification AUTOSAR_SWS_UpdateConfigurationManagementMaster, 2022. <https://www.autosar.org/>